ניהול צבא של סוכנים

ארכיטקטורת עץ פורש בינארי למערכות ענק

Managing an Army of Agents:
Binary Spanning Tree Architecture for Huge Systems

Author: Dr. Rami Segal

All rights reserved

January 2026

תוכן העניינים

1. מבוא לתזמור סוכנים 1
1.1 מדוע אנחנו צריכים לנהל סוכנים מרובים? 1
1.1.1 האתגר המרכזי 1
2.1.1 הפתרון: פירוק וריכוז 1
2.1 הרעיון המרכזי: עץ פורש בינארי 2
3.1 יתרונות הארכיטקטורה 2
1.1.3 הגדלת Context Window האפקטיבי 2
2.1.3 עבודה במקביל 2
3.1.3 בקרה ומעקב 2
4.1 מבנה הספר 3
5.1 למי מיועד ספר זה? 3

2. ארכיטקטורת עץ פורש בינארי 5
1.2 מבנה העץ הבסיסי 5
1.2.1 הגדרות בסיסיות 5
2.2 חישוב מספר הסוכנים 5
3.2 מבנה העץ המורחב 6
1.2.3 מתן שמות לצמתים 6
4.2 תקשורת בין צמתים 6
1.2.4 זרימת מידע מלמעלה למטה 6
2.2.4 זרימת תוצאות מלמטה למעלה 6
5.2 תיקיית Shared --- משאבים משותפים 7
6.2 יתרונות ארכיטקטורת BST 7

3. ניהול טוקנים וחלונות הקשר 9
1.3 מהם טוקנים? 9
1.3.1 מגבלות טוקנים 9
2.3 האתגר: מגבלת הטוקנים בפועל 9
1.3.2 הכלל המעשי 01
3.3 חישוב תקציב טוקנים 01
1.3.3 תקציב חודשי 01
2.3.3 חלוקה לפי סוכנים 01
4.3 אסטרטגיות לחיסכון בטוקנים 01
1.3.4 פירוק משימות 01
2.3.4 שימוש בתבניות 01
3.3.4 מזעור הקשר 01
5.3 ניטור ובקרה 11
1.3.5 כלים למעקב 11

4. אסטרטגיות הרצה מקבילית 31

1.4 למה הרצה מקבילית? 31
1.4.1 היתרון המרכזי 31
2.4 מודל 16 הטרמינלים 31
1.4.2 למה דווקא 16? 41
3.4 סנכרון בין סוכנים 41
1.4.3 תלויות בין משימות 41
2.4.3 אסטרטגיות סנכרון 41
4.4 טיפול בשגיאות 41
1.4.4 מה קורה כשסוכן נכשל? 41
2.4.4 מנגנון Retry 51
5.4 איזון עומסים 51
1.4.5 בעיית האיזון 51
2.4.5 פתרון: חלוקה דינמית 51
6.4 מדדי ביצועים 51

5. ארגון היררכי של מודולים 71
1.5 עקרונות הארגון ההיררכי 71
1.5.1 מבנה התיקיות 71
2.5 תיקיית Specs --- המפרט 81
1.5.2 חשיבות ה-PRD 81
3.5 תיקיות Shared --- משאבים משותפים 81
1.5.3 היררכיית Shared 81
2.5.3 כללים לשימוש ב-Shared 81
4.5 מודולי Worker --- ביצוע העבודה 91
1.5.4 מה הם Workers? 91
2.5.4 מבנה Worker 91
5.5 תקשורת בין רבדים 91
1.5.5 ממשקים מוגדרים 91
2.5.5 תקשורת באמצעות הודעות 91
6.5 סיכום הארגון ההיררכי 02

6. דפוס הפירוק ל-63 מודולים 12
1.6 מדוע דווקא 63? 12
1.6.1 השיקולים שהובילו ל-63 12
2.6 מבנה העץ של 63 מודולים 22
3.6 תהליך הפירוק 22
1.6.3 שלב 1: הגדרת ה-PRD 22
2.6.3 שלב 2: פירוק ראשוני לתחומים 22
3.6.3 שלב 3: פירוק מפורט 32
4.6 חלוקת העבודה על 16 טרמינלים 32
1.6.4 אלגוריתם החלוקה 32
2.6.4 דוגמה לחלוקה 42
5.6 ניהול תלויות בין מודולים 42
1.6.5 גרף תלויות 42
2.6.5 סדר ביצוע 42
6.6 מסקנות 42

7. מתודולוגיית פיתוח 52
1.7 מחזור החיים של פרויקט BST 52

2.7 שלב 1: כתיבת ה-PRD 52
1.7.2 מה כולל PRD טוב? 52
2.7.2 תבנית PRD 52
3.7 שלב 2: תכנון הארכיטקטורה 62
1.7.3 בניית התוכנית 62
4.7 שלב 3: פירוק למודולים 62
1.7.4 כללים לפירוק 62
2.7.4 תהליך הפירוק 72
5.7 שלב 4: בנייה מקבילית 72
1.7.5 הפעלת הסוכנים 72
2.7.5 ניטור ההתקדמות 72
6.7 שלב 5: בדיקות 82
1.7.6 רמות הבדיקה 82
7.7 שלב 6: אינטגרציה 82
1.7.7 איחוד המודולים 82
8.7 שלב 7: Debug וניפוי שגיאות 82
1.7.8 זיהוי בעיות 82
2.7.8 תיקון שגיאות 82
9.7 מדדי הצלחה 92

8. מימוש מעשי ומקרי בוחן 13
1.8 מקרה בוחן: מערכת עיבוד חשבוניות 13
1.8.1 הגדרת הבעיה 13
2.8.1 ה-PRD 13
2.8 הפירוק למודולים 23
1.8.2 מבנה העץ 23
2.8.2 רשימת המודולים לפי תחום 23
3.8 מימוש מודול לדוגמה 23
1.8.3 מודול קריאת IMAP 23
4.8 הפעלת 16 טרמינלים 33
1.8.4 סקריפט הפעלה 33
5.8 ניטור ובקרה 43
1.8.5 לוח בקרה 43
6.8 תוצאות 43
1.8.6 מדדי הצלחה 43
7.8 לקחים ותובנות 53
1.8.7 מה עבד טוב 53
2.8.7 מה ניתן לשפר 53
8.8 המלצות ליישום 53
9.8 סיכום 53
01.8 מבט לעתיד 63

9. מטלת בית: אופטימיזציה, מודולריות ובקרת עומסים באורקסטרציית BST 73
1.9 מטרת המטלה 73
2.9 דרישות מבניות: עץ ה-PRD הבינארי (Spec Tree) 73
1.9.2 עומק ומבנה 73
2.9.2 צומת כיחידה עצמאית (Standalone Mini-Project) 73
3.9.2 חוק העלים 83

© Dr. Rami Segal - כל הזכויות שמורות

3.9 מתודולוגיית פיתוח: Bottom-Up Workflow 83
4.9 בקרת עומסים וניתוח מתמטי 83
1.9.4 אלגוריתם בקרת עומסים 83
2.9.4 ניתוח מורכבות (*O* Notation) 83
3.9.4 סקר ספרות 83
5.9 סימולציית תרחישים (10 תרחישים) 83
6.9 דרישות הגשה 93
1.9.6 GitHub Repository 93
2.9.6 דוחות כיסוי קוד 93
3.9.6 מפת העץ 93
4.9.6 דוח טכני 93

א'. אופטימיזציית הקצאת סוכנים בארכיטקטורת BST 14
א'.1 מבוא לבעיית האופטימיזציה 14
1.א'.1 הגדרת הבעיה 14
א'.2 הגדרת הפרמטרים 14
1.א'.2 סוגי הסוכנים 14
2.א'.2 מבנה העץ 24
א'.3 ניסוח בעיית התכנות הליניארי 24
1.א'.3 משתני ההחלטה 24
2.א'.3 פונקציית המטרה 24
3.א'.3 אילוצים 34
א'.4 פתרון באמצעות Python 44
1.א'.4 קוד הפתרון 44
2.א'.4 תוצאות הפתרון 54
3.א'.4 ויזואליזציה של ההקצאה 64
א'.5 ניתוח רגישות 64
1.א'.5 רגישות לתקציב האנרגיה 64
2.א'.5 רגישות למגבלת זמן העיבוד 84
3.א'.5 רגישות לכמות הסוכנים 84
א'.6 מסקנות ותובנות 94
1.א'.6 תובנות מהאופטימיזציה 94
2.א'.6 המלצות ליישום 94
3.א'.6 הרחבות אפשריות 94
א'.7 קוד מלא להרצה 94
מקורות 25

פרק 1

מבוא לתזמור סוכנים

1.1 מדוע אנחנו צריכים לנהל סוכנים מרובים?

לאורך ההיסטוריה האנושית, כל פריצת דרך גדולה בייצור התרחשה כאשר למדנו לתאם מאמצים בין ישויות רבות. המהפכה החקלאית לימדה אותנו לארגן משאבי טבע; המהפכה התעשייתית הראתה לנו כיצד לארגן מכונות; והמהפכה הדיגיטלית איפשרה לנו לתאם בין מעבדים. כעת אנו עומדים בפתחה של מהפכה חדשה --- היכולת לתאם בין סוכנים אינטליגנטיים.

בעולם הפיתוח המודרני, אנו עומדים בפני אתגר משמעותי: כיצד ניתן לבנות מערכות תוכנה מורכבות בזמן קצר? התשובה המסורתית הייתה להעסיק יותר מתכנתים, אך גישה זו מוגבלת מטבעה. פרד ברוקס הראה לנו בספרו המיתולוגי ``The Mythical Man-Month'' שהוספת מתכנתים לפרויקט מאחר רק מאחרת אותו עוד יותר.

כיום, עם התפתחות מודלי השפה הגדולים (Large Language Models - LLMs), נפתח בפנינו אופק חדש: היכולת להפעיל **צבא של סוכני AI** שעובדים במקביל על משימות פיתוח. אך עם כוח זה מגיעה גם אחריות. אנו לא רק מפתחים תוכנה --- אנו מעצבים את הדרך שבה בני אדם יתקשרו עם מכונות חושבות בעתיד.

1.1.1 האתגר המרכזי

הבעיה המרכזית שעומדת בפנינו היא **מגבלת הטוקנים**. כל מודל שפה מוגבל בכמות המידע שהוא יכול לעבד בו-זמנית --- מה שמכונה Context Window. ניתן לדמות זאת לזיכרון העבודה של אדם: גם המוח האנושי מוגבל ביכולתו להחזיק כמה פריטי מידע בו-זמנית, ולכן אנו משתמשים בכתיבה, בתרשימים ובכלים חיצוניים.

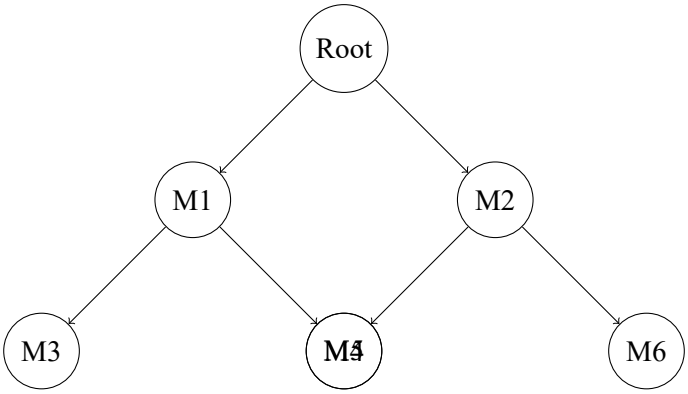
כאשר אנו מנסים לתת למודל משימה גדולה מדי, המודל מתחיל לשכוח חלקים מהמשימה, הפלט הופך לא עקבי, ואיכות העבודה יורדת באופן משמעותי. תופעה זו אינה פגם בטכנולוגיה --- היא מגבלה מהותית של מערכות עיבוד מידע, בדיוק כפי שזיכרון אנושי מוגבל. הפתרון, כפי שנראה, טמון לא בהגדלת הזיכרון אלא בארגון חכם של העבודה.

1.1.2 הפתרון: פירוק וריכוז

הפתרון שנציג בספר זה מבוסס על עיקרון פשוט אך עוצמתי: **פירוק המשימה למשימות קטנות** והפעלת סוכנים מרובים במקביל.

במקום לתת לסוכן אחד לכתוב 4000 שורות קוד, אנו מחלקים את העבודה בין 16 סוכנים, כך שכל אחד מטפל בכ-250 שורות בלבד.

1.2 הרעיון המרכזי: עץ פורש בינארי



איור 1.1: מבנה בסיסי של עץ פורש בינארי

הארכיטקטורה שנציג מבוססת על **עץ פורש בינארי** (Binary Spanning Tree), כפי שמוצג באיור 1.1. מבנה זה אינו אקראי --- הוא משקף את אחד הדפוסים היעילים ביותר בטבע ובמערכות מידע לניהול היררכי. בעץ זה, השורש משמש כסוכן המנהל שמקבל את המשימה הגדולה ומחלק אותה לחלקים קטנים יותר. הצמתים הפנימיים הם סוכנים שמנהלים תת-עצים, ממשיכים את תהליך החלוקה עד שהמשימות הופכות לניתנות לביצוע. בתחתית העץ נמצאים העלים --- הסוכנים שמבצעים את העבודה בפועל, כותבים קוד, מריצים בדיקות, ומעבדים נתונים.

הערה חשובה

בכל מקום בספר בו מוזכר צומת בעץ, הכוונה היא למקום שיכול להיות יחידת קוד עצמאית ואוטונומית (כגון סוכן AI) המתקשרת עם הילדים והאב בלבד.

1.3 יתרונות הארכיטקטורה

1.3.1 הגדלת Context Window האפקטיבי

כל סוכן פועל עם Context Window עצמאי. כאשר מפעילים 16 סוכנים במקביל, אנו מקבלים פי 16 יותר ``זיכרון עבודה" לעיבוד מידע.

1.3.2 עבודה במקביל

במקום עבודה בטור, כל הסוכנים יכולים לעבוד בו-זמנית. משימה שהייתה לוקחת שבועות --- יכולה להסתיים בימים בודדים.

1.3.3 בקרה ומעקב

הארכיטקטורה ההיררכית מספקת יתרון קריטי נוסף: יכולת בקרה ומעקב. כמו מפקד צבאי שמנהל את כוחותיו דרך שרשרת פיקוד, אנו יכולים לעקוב אחר התקדמות כל סוכן, לזהות צווארי בקבוק לפני שהם הופכים לבעיות קריטיות, ולבצע איזון עומסים דינמי כאשר סוכנים מסוימים מתקשים. שקיפות זו היא מפתח להצלחה בניהול מערכות מורכבות, ומאפשרת לנו להתערב בזמן כשמשהו משתבש.

© Dr. Rami Segal - כל הזכויות שמורות

1.4 מבנה הספר

ספר זה בנוי כמסע מן התיאוריה אל הפרקטיקה. נפתח בפרק 2 בהצגת הארכיטקטורה המלאה של עץ פורש בינארי, כולל המתמטיקה שמאחוריה והסיבות שהובילו לבחירה במבנה זה. בפרק 3 נצלול לתוך אחד המשאבים הקריטיים ביותר --- הטוקנים --- ונלמד כיצד לנהל אותם בחכמה.

לאחר הבנת היסודות, פרק 4 יציג אסטרטגיות להרצה מקבילית, כולל טיפול בסנכרון ובשגיאות. פרק 5 ידון בארגון היררכי של מודולים ובתפקיד תיקיות ה-Shared. בפרק 6 נציג את דפוס הפירוק ל-63 מודולים --- מספר שנבחר בקפידה על סמך שיקולים מעשיים ותיאורטיים.

בחלק האחרון, פרק 7 יציג את מתודולוגיית הפיתוח המלאה, מכתיבת דרישות ועד פריסה. לבסוף, פרק 8 יחבר את כל החלקים יחד דרך מקרה בוחן מעשי.

1.5 למי מיועד ספר זה?

ספר זה נכתב עבור שלושה קהלים שונים אך משלימים זה את זה. **המהנדס הבכיר** ימצא כאן ארכיטקטורה מעשית ליישום מיידי, כולל דפוסי עיצוב, קוד לדוגמה, ושיקולי ביצועים. **החוקר האקדמי** יגלה שאלות פתוחות מרתקות בתחומי מערכות מבוזרות, תיאום סוכנים, ואופטימיזציה. **הסטודנט** יקבל מבוא מקיף לעולם חדש, עם דגש על עקרונות יסוד ואנלוגיות שמקלות על ההבנה.

בכל פרק הוקדשה תשומת לב לשלושת הקהלים: התיאוריה מוסברת לעומק, אך תמיד מלווה בדוגמאות מעשיות ובקוד שניתן להריץ. ההשלכות הרחבות יותר --- האתיות, הפילוסופיות והחברתיות --- משולבות בתוך הדיון הטכני, שכן אי אפשר להפריד בין השניים.

בפרק הבא נצלול לעומק הארכיטקטורה של עץ פורש בינארי, נבין את המתמטיקה שמאחוריה, ונראה כיצד מבנה פשוט זה מאפשר ניהול של עשרות סוכנים במקביל.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 2

ארכיטקטורת עץ פורש בינארי

2.1 מבנה העץ הבסיסי

בטבע, מבנים היררכיים מופיעים שוב ושוב: ענפי העץ, מערכת העצבים, מבנה הברונכיות בריאות. האבולוציה ``גילתה" שמבנה עץ הוא דרך יעילה להפיץ מידע או משאבים ממקור מרכזי אחד אל נקודות קצה רבות. אין זה מקרי שאנו משתמשים באותו מבנה בדיוק לניהול סוכני AI.

ארכיטקטורת עץ פורש בינארי (Binary Spanning Tree - BST) היא הבסיס לניהול יעיל של סוכני AI מרובים. העץ מאפשר חלוקה היררכית של משימות, כאשר כל צומת אחראי על תת-עץ של סוכנים. יתרונו הגדול של המבנה הבינארי הוא פשטותו: כל צומת מקבל לכל היותר שני בנים, דבר המפשט את הלוגיקה של חלוקה ואיחוד.

2.1.1 הגדרות בסיסיות

לפני שנצלול לעומק הארכיטקטורה, נגדיר את המונחים המרכזיים. השורש (Root) הוא הסוכן המרכזי שמקבל את המשימה הראשית ומחלק אותה לתת-משימות. צומת אב (Parent Node) הוא כל סוכן שמנהל סוכנים אחרים מתחתיו, בעוד צומת בן (Child Node) הוא סוכן שמקבל משימות מסוכן האב שלו. העלים (Leaves) הם הסוכנים בתחתית העץ שמבצעים עבודה בפועל, ללא סוכנים מתחתם. עומק העץ (Tree Depth) מציין את מספר הרמות מהשורש עד לעלים.

2.2 חישוב מספר הסוכנים

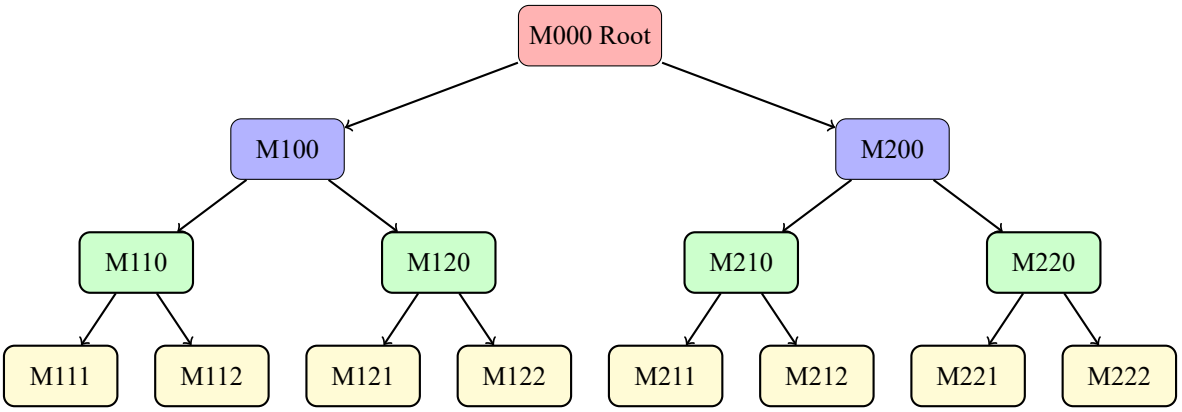
המתמטיקה של עץ בינארי מלא היא אלגנטית ופשוטה. בעץ בינארי מלא עם d רמות, מספר הצמתים הכולל הוא:

$$N = 2^d - 1 \tag{2.1}$$

נוסחה זו נובעת מסכום הסדרה הגיאומטרית $1 + 2 + 4 + \ldots + 2^{d-1}$. למעשה, כל רמה בעץ מכילה בדיוק פעמיים את מספר הצמתים של הרמה שלפניה, מה שמוביל לצמיחה אקספוננציאלית. עובדה זו היא גם היתרון וגם האתגר של מבנה זה.

בפועל, נוסחה 2.1 נותנת לנו עץ עם 3 רמות שמכיל 7 סוכנים, עץ עם 4 רמות שמכיל 15 סוכנים, ועץ עם 6 רמות שמכיל 63 סוכנים. בחירת העומק הנכון תלויה בגודל הפרויקט, בתקציב הזמין וביכולת הניהול --- נושא שנרחיב עליו בפרק 6.

2.3 מבנה העץ המורחב



איור 2.1: מבנה עץ BST עם 4 רמות

איור 2.1 מציג עץ בינארי עם 4 רמות. שימו לב לקידוד הצבעים: השורש (באדום) מנהל את המערכת כולה, הצמתים ברמה הראשונה (בכחול) מנהלים תחומים גדולים, הרמה השנייה (בירוק) מנהלת תת-תחומים, והעלים (בצהוב) מבצעים את העבודה בפועל.

2.3.1 מתן שמות לצמתים

מערכת שמות עקבית היא קריטית לניהול עץ סוכנים. אימצנו קונבנציה שבה כל צומת מקבל מזהה ייחודי לפי מיקומו בעץ. השורש נקרא `M000`, בניו הישירים הם `M100` ו-`M200`, ובניהם של אלה הם `M110`, `M120`, `M210`, ו-`M220`. השיטה ממשיכה באופן דומה לכל עומק נוסף. קידוד זה מאפשר לזהות מיידית את מיקומו של כל סוכן בהיררכיה, ולהבין את יחסי האב-בן ללא צורך במפה נוספת.

2.4 תקשורת בין צמתים

2.4.1 זרימת מידע מלמעלה למטה

1. **המשימה הראשית** מגיעה לשורש
2. **השורש מפרק** את המשימה לתת-משימות
3. **כל בן** מקבל תת-משימה ייחודית
4. **הבנים ממשיכים לפרק** עד שמגיעים לעלים

2.4.2 זרימת תוצאות מלמטה למעלה

1. **העלים מבצעים** את העבודה בפועל
2. **התוצאות עולות** לאבות
3. **כל אב מאחד** את תוצאות הבנים
4. **השורש מקבל** את התוצאה המאוחדת הסופית

© Dr. Raimund Seidel - כל הזכויות שמורות

2.5 תיקיית Shared --- משאבים משותפים

כל רמה בעץ יכולה לכלול תיקיית shared המכילה משאבים משותפים לכל הצמתים ברמה זו:

```
project/
├── shared/              # Shared for all levels
│   ├── types.py
│   └── config.py
├── level1/
│   ├── shared/          # Shared for level 1
│   │   └── utils.py
│   ├── M100/
│   └── M200/
└── level2/
    ├── shared/          # Shared for level 2
    │   └── helpers.py
    ├── M110/
    └── M120/
```

2.6 יתרונות ארכיטקטורת BST

ארכיטקטורת עץ פורש בינארי מציעה מספר יתרונות משמעותיים שהופכים אותה לבחירה הטבעית לניהול סוכנים. ראשית, הסקלביליות: קל להוסיף רמות וסוכנים לפי הצורך, ללא שינוי מהותי בארכיטקטורה. שנית, הבידוד: כל סוכן עובד עם Context Window עצמאי, מה שמונע ``זיהום" בין משימות שונות.

שלישית, העקיבות: המבנה ההיררכי מאפשר לעקוב בקלות אחר מי עשה מה, ולזהות בעיות במקורן. רביעית, הגמישות: ניתן להתאים את עומק העץ לפרויקט הספציפי, משבעה סוכנים לפרויקט קטן ועד עשרות סוכנים לפרויקט גדול. ולבסוף, המקביליות: כל הסוכנים ברמה נתונה יכולים לעבוד בו-זמנית, מה שמאפשר האצה דרמטית של תהליך הפיתוח.

יתרונות אלה אינם רק תיאורטיים --- הם משתקפים בתוצאות מעשיות, כפי שנראה במקרה הבוחן בפרק 8.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 3

ניהול טוקנים וחלונות הקשר

3.1 מהם טוקנים?

אם נתבונן במודלי שפה כמפעל, הטוקנים הם חומר הגלם שלו. כמו שמפעל מתכת יכול לעבד כמות מוגבלת של פלדה בכל יום, כך גם מודל שפה מוגבל בכמות הטוקנים שהוא יכול לעבד בכל פנייה. הבנת המגבלה הזו, והיכולת לעבוד בתוכה, היא המפתח להצלחה בניהול צבא של סוכנים.

טוקנים (Tokens) הם יחידות הבסיס שמודלי שפה משתמשים בהן לעיבוד טקסט. הם אינם מילים במובן הרגיל, אלא יחידות שנקבעות על ידי אלגוריתם הטוקניזציה. מילה באנגלית מתורגמת בדרך כלל לטוקן אחד או שניים, בעוד שמילה בעברית עשויה לדרוש שלושה עד חמישה טוקנים בשל מורכבות השפה.

3.1.1 מגבלות טוקנים

כל מודל שפה מוגבל בכמות הטוקנים שהוא יכול לעבד בכל פנייה. טבלה 3.1 מציגה את חלונות ההקשר של מודלים נפוצים.

מודל	Context Window	הערות
GPT-4	8K / 32K	גרסאות שונות
Claude 3	200K	חלון גדול
Claude Code	200K	מותאם לקוד

טבלה 3.1: חלונות הקשר של מודלי שפה שונים

מספרים אלה עשויים להיראות גדולים, אך בפועל הם מתמלאים במהירות מפתיעה. הנחיות מערכת, היסטוריית שיחה, קוד מקור, ותיעוד --- כל אלה צורכים טוקנים. לכן, למרות שחלון של 200K טוקנים נשמע עצום, בפועל אנו צריכים לנהל אותו בזהירות.

3.2 האתגר: מגבלת הטוקנים בפועל

למרות שחלונות ההקשר גדלו משמעותית בשנים האחרונות, בפועל אנו עדיין נתקלים במגבלות משמעותיות (כפי שצוין בפרק 1). יותר טוקנים פירושם עלות גבוהה יותר --- הן מבחינה כספית והן מבחינת משאבי מחשוב. בנוסף, עיבוד טקסט ארוך לוקח יותר זמן, מה שמאט את קצב העבודה.

התובנה המרכזית היא שהפתרון אינו לחכות לחלונות הקשר גדולים יותר. גם אם יפותחו מודלים עם חלונות של מיליוני טוקנים, הבעיות הללו יישארו רלוונטיות. הפתרון האמיתי הוא עיצוב מערכות שעובדות היטב בתוך המגבלות הקיימות.

3.2.1 הכלל המעשי

כלל אצבע: משימות קטנות יותר = תוצאות טובות יותר.

כאשר נותנים לסוכן משימה ממוקדת וקצרה, הוא מבצע אותה בצורה מדויקת יותר מאשר משימה גדולה ומורכבת.

3.3 חישוב תקציב טוקנים

3.3.1 תקציב חודשי

נניח תקציב של $200 לחודש לשימוש ב-Claude Code:

$$\text{טוקנים זמינים} = \frac{\$200}{\text{עלות לטוקן}} \tag{3.1}$$

3.3.2 חלוקה לפי סוכנים

אם מפעילים 16 טרמינלים במקביל:

$$\text{טוקנים לסוכן} = \frac{\text{סה"כ טוקנים}}{16} \tag{3.2}$$

3.4 אסטרטגיות לחיסכון בטוקנים

3.4.1 פירוק משימות

במקום:

``כתוב לי תוכנה שלמה לניהול חשבונות"

עדיף:

``כתוב מודול לקריאת קבצי CSV"

3.4.2 שימוש בתבניות

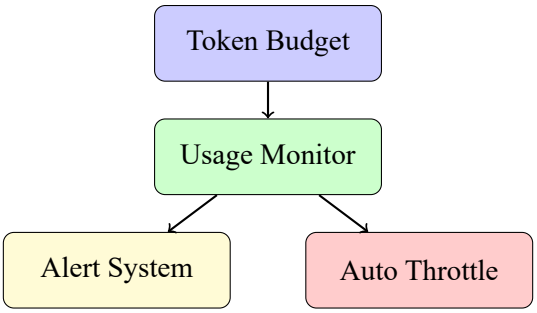
יצירת תבניות קוד משותפות שכל הסוכנים משתמשים בהן חוסכת טוקנים רבים.

3.4.3 מזעור הקשר

שליחת רק המידע הנחוץ לכל משימה, במקום ``לזכור" את כל ההיסטוריה.

© Dr. Rami Segal - כל הזכויות שמורות

3.5 ניטור ובקרה



איור 3.1: מערכת ניטור טוקנים

איור 3.1 מציג את הארכיטקטורה הבסיסית של מערכת ניטור טוקנים. תקציב הטוקנים זורם למערכת הניטור, המפיצה התראות כאשר מתקרבים לגבול ומפעילה מנגנון חסימה אוטומטי כאשר חורגים ממנו.

3.5.1 כלים למעקב

ניטור יעיל דורש שלושה רכיבים משלימים. לוגים מפורטים רושמים כל פנייה וכמות הטוקנים שנצרכו, ומאפשרים ניתוח בדיעבד של דפוסי השימוש. מערכת התראות שולחת הודעה כשמתקרבים לגבול התקציב, ומאפשרת התערבות מניעתית. ולבסוף, מנגנון חסימה אוטומטית עוצר סוכנים שצורכים יותר מהמוקצה להם, ומונע חריגה מהתקציב הכולל.

בפרק הבא נעסוק באסטרטגיות הרצה מקבילית --- כיצד לגרום למספר סוכנים לעבוד בו-זמנית, לסנכרן ביניהם, ולהתמודד עם שגיאות.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 4

אסטרטגיות הרצה מקבילית

4.1 למה הרצה מקבילית?

ההיסטוריה של המחשוב היא במידה רבה היסטוריה של מקביליות. מהמעבד הראשון שהריץ פקודה אחת בכל פעם, דרך מעבדים מרובי ליבות, ועד לעיבוד מבוזר בענן --- התקדמנו על ידי מציאת דרכים לעשות יותר דברים בו-זמנית. ניהול צבא של סוכני AI הוא הצעד הבא בהתפתחות זו.

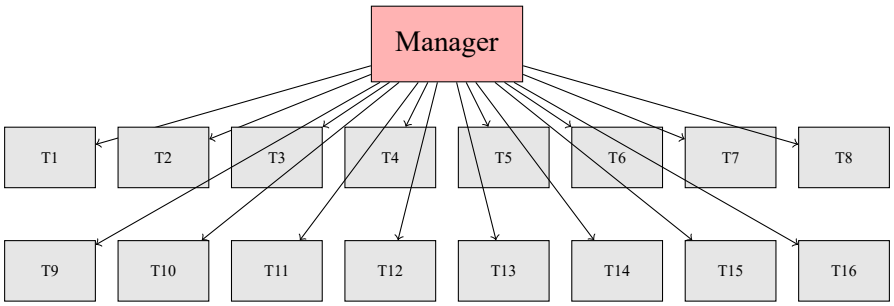
כפי שהוצג בפרק 1, הרצה מקבילית היא המפתח להאצת תהליך הפיתוח. עיקרון זה פשוט, אך המימוש מורכב: כיצד מחלקים את העבודה? כיצד מתאמים בין הסוכנים? ומה עושים כשמשהו משתבש?

4.1.1 היתרון המרכזי

אם משימה לוקחת 16 שעות לסוכן אחד, הפעלת 16 סוכנים במקביל יכולה להפחית את הזמן לכשעה אחת בלבד (בתנאים אידאליים).

$$T_{parallel} = \frac{T_{sequential}}{N_{agents}} \tag{4.1}$$

4.2 מודל 16 הטרמינלים



איור 4.1: מודל 16 הטרמינלים

איור 4.1 מציג את מודל 16 הטרמינלים. המנהל המרכזי מחלק משימות ל-16 סוכנים עצמאיים.

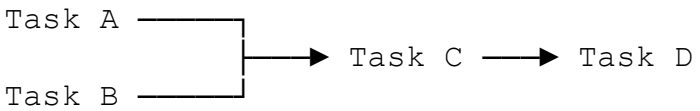
4.2.1 למה דווקא 16?

המספר 16 לא נבחר באקראי --- הוא תוצאה של שיקולים מעשיים מרובים שמתכנסים לאותה נקודה. מבחינת תקציב, כאשר מקצים $200 לפרויקט, חלוקה ל-16 סוכנים נותנת כ-$12.5 לכל אחד, סכום שמספיק למשימה מורכבת אך לא מבזבז משאבים. מבחינת יכולת ניהול, 16 סוכנים הם מספר שאדם אחד יכול לעקוב אחריהם במקביל ללא עומס קוגניטיבי מוגזם. מבחינת מגבלות מערכת, הפעלת יותר מ-16 טרמינלים בו-זמנית מתחילה להעמיס על משאבי המחשב. ולבסוף, מבחינת איזון משימות, 63 מודולים מתחלקים היטב על 16 סוכנים לכדי 4 מודולים לסוכן בממוצע.

4.3 סנכרון בין סוכנים

4.3.1 תלויות בין משימות

לא כל המשימות יכולות לרוץ במקביל. ישנן משימות שתלויות בתוצאות של משימות אחרות.



4.3.2 אסטרטגיות סנכרון

קיימות שלוש אסטרטגיות עיקריות לסנכרון בין סוכנים, כל אחת מתאימה לסיטואציות שונות. האסטרטגיה הראשונה, Barrier Sync, דורשת מכל הסוכנים לחכות בנקודה מסוימת עד שכולם מגיעים. זו השיטה הפשוטה ביותר, אך עלולה לגרום לבזבוז זמן כשסוכנים מהירים מחכים לאיטיים.

האסטרטגיה השנייה, Message Passing, מאפשרת לסוכנים להעביר הודעות ביניהם ישירות. זו שיטה גמישה יותר אך דורשת תשתית מורכבת יותר. האסטרטגיה השלישית, Shared State, משתמשת בתיקיית shared משותפת שבה סוכנים כותבים וקוראים קבצים. שיטה זו פשוטה ליישום ומתאימה במיוחד לשיתוף קוד משותף.

4.4 טיפול בשגיאות

4.4.1 מה קורה כשסוכן נכשל?

כאשר סוכן נכשל במשימתו:

1. **זיהוי** --- המנהל מזהה את הכשלון
2. **בידוד** --- הסוכן הכושל מבודד מהמערכת
3. **ניסיון חוזר** --- המשימה מוקצית לסוכן אחר
4. **דיווח** --- הכשלון נרשם לצורך ניתוח

© Dr. Raz Lies - כל הזכויות שמורות

4.4.2 מנגנון Retry

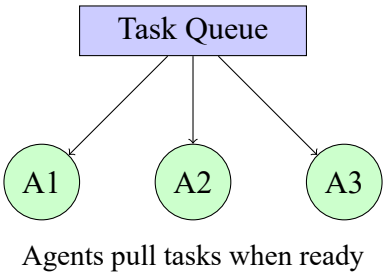
```
MAX_RETRIES = 3

def execute_with_retry(task, agent):
    for attempt in range(MAX_RETRIES):
        try:
            result = agent.execute(task)
            return result
        except AgentError as e:
            log_error(f"Attempt {attempt+1} failed: {e}")
            if attempt == MAX_RETRIES - 1:
                raise
    return None
```

4.5 איזון עומסים

4.5.1 בעיית האיזון

לא כל המשימות שוות בגודלן. סוכן שמקבל משימה קטנה יסיים מהר ויחכה, בעוד סוכן עם משימה גדולה יעכב את כולם.

4.5.2 פתרון: חלוקה דינמית



איור 4.2: חלוקה דינמית של משימות

איור 4.2 מציג את עקרון החלוקה הדינמית. במקום להקצות מראש משימות לסוכנים, קיים תור משימות מרכזי. סוכן שמסיים את משימתו הנוכחית ``מושך" את המשימה הבאה מהתור. שיטה זו מבטיחה שסוכנים מהירים מקבלים יותר משימות, וסוכנים איטיים אינם מעכבים את המערכת.

4.6 מדדי ביצועים

כדי לשפר ביצועים, ראשית צריך למדוד אותם. ארבעה מדדים מרכזיים קובעים את הצלחת ההרצה המקבילית. Throughput מודד כמה משימות מושלמות ליחידת זמן --- זהו המדד הישיר ביותר לפרודוקטיביות. Latency מודד את הזמן מקבלת משימה עד השלמתה --- חשוב במיוחד כשיש תלויות בין משימות. Utilization מודד את אחוז הזמן שהסוכנים עובדים בפועל ולא מחכים --- יעילות נמוכה מעידה על בעיות באיזון העומסים. ולבסוף, Token Efficiency מודד כמה טוקנים נצרכים למשימה --- מדד קריטי לניהול תקציב.

© Dr. Rami Segal - כל הזכויות שמורות

שילוב של ארבעת המדדים הללו נותן תמונה מלאה של ביצועי המערכת ומאפשר זיהוי צווארי בקבוק.

בפרק הבא נעסוק בארגון היררכי של מודולים, שהוא המפתח לפירוק יעיל של משימות.

כל הזכויות שמורות - Dr. Rami Segal ©

פרק 5

ארגון היררכי של מודולים

5.1 עקרונות הארגון ההיררכי

אחד הלקחים החשובים ביותר מתורת המורכבות הוא שמערכות מורכבות מאוד הן כמעט תמיד היררכיות. הרברט סיימון, חתן פרס נובל, טען שהיררכיה היא ``ארכיטקטורת המורכבות" --- הדרך שבה מערכות מורכבות מתהוות ונשארות יציבות. הסיבה פשוטה: מערכת היררכית יכולה להתפתח בהדרגה, כאשר כל רמה ``עומדת על כתפי" הרמה שמתחתיה.

ארגון היררכי של מודולים מאפשר ניהול יעיל של פרויקטים גדולים. הרעיון המרכזי הוא לחלק את המערכת לרבדים, כאשר כל רובד מכיל רכיבים ברמת הפשטה דומה. הרובד העליון מגדיר ``מה" צריך לקרות, והרבדים שמתחתיו מגדירים ``איך". עיקרון זה, המוכר מעולם הארכיטקטורה התוכנתית, מתאים במיוחד לניהול סוכנים.

5.1.1 מבנה התיקיות

```
project/
├── specs/                  # High-level specifications
│   ├── PRD.md              # Product Requirements
│   └── architecture.md     # System architecture
├── shared/                 # Shared across all modules
│   ├── types/
│   ├── utils/
│   └── config/
├── layer1/                 # Top-level modules
│   ├── shared/             # Shared within layer 1
│   ├── module_a/
│   └── module_b/
├── layer2/                 # Mid-level modules
│   ├── shared/
│   ├── module_c/
│   └── module_d/
└── layer3/                 # Worker modules
    ├── shared/
    ├── worker_1/
    └── worker_2/
```

5.2 תיקיית Specs --- המפרט

תיקיית Specs היא ``מוחו" של הפרויקט. היא מכילה את כל המידע הנחוץ להבנת הפרויקט ברמה העליונה. ה-PRD (מסמך דרישות המוצר) מגדיר מה המערכת צריכה לעשות ולמה. מסמך הארכיטקטורה מתאר את המבנה הכללי ואת השיקולים שהובילו אליו. מפרטי ה-API מגדירים כיצד מודולים שונים מתקשרים ביניהם. ותרשימי זרימת המידע מראים כיצד נתונים עוברים דרך המערכת.

חשיבותה של תיקייה זו חורגת מתיעוד גרידא. היא משמשת כמקור האמת המרכזי שאליו כל הסוכנים פונים כשיש להם שאלות. כאשר סוכן אינו בטוח מה לעשות, הוא קודם כל בודק את ה-PRD. כאשר יש אי-הבנה בין סוכנים, מפרטי הממשקים מכריעים.

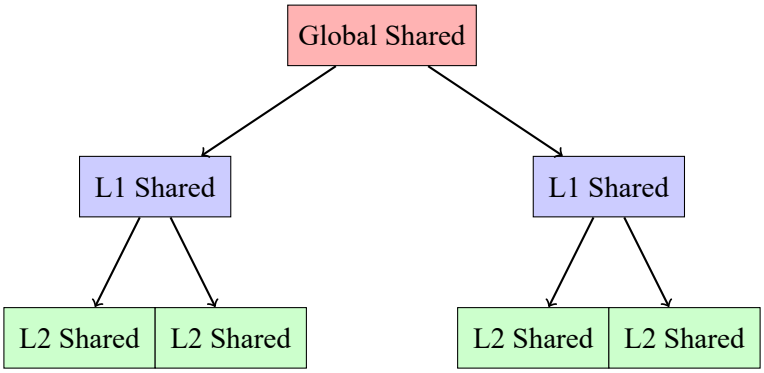
5.2.1 חשיבות ה-PRD

ה-PRD הוא המסמך המרכזי שממנו נגזר כל הפרויקט. הוא מגדיר:

1. **מה הבעיה** שהמערכת פותרת
2. **מי המשתמשים** ומה הצרכים שלהם
3. **מה הפתרון** המוצע
4. **מה הקריטריונים** להצלחה

5.3 תיקיות Shared --- משאבים משותפים

5.3.1 היררכיית Shared



איור 5.1: היררכיית תיקיות Shared

איור 5.1 מציג את היררכיית תיקיות ה-Shared. שימו לב לכיוון החיצים: תיקיית Shared גלובלית משרתת את כל הרבדים, בעוד כל רובד יכול להוסיף Shared משלו לשימוש פנימי.

5.3.2 כללים לשימוש ב-Shared

ארבעה כללים מנחים את השימוש בתיקיות Shared. ראשית, קוד משותף מועבר תמיד לתיקיית ה-Shared הרלוונטית --- ברמה הנמוכה ביותר שעדיין משרתת את כל המשתמשים בו. שנית, אין כפילויות: אם קוד מופיע פעמיים, זוהי סיבה להעבירו ל-Shared. שלישית, תלות היא תמיד חד-כיוונית: מודולים תלויים ב-Shared, אך Shared לעולם אינו תלוי במודולים ספציפיים. ורביעית, גודל מינימלי: רק מה שבאמת משותף נכנס ל-Shared, לא ``מה שאולי יהיה שימושי בעתיד".

© Dr. Rami Segal - כל הזכויות שמורות

5.4 מודולי Worker --- ביצוע העבודה

5.4.1 מה הם Workers?

כפי שהוסבר בפרק 2, מודולי Worker הם העלים בעץ שמבצעים את העבודה בפועל. הם כותבים קוד, מריצים בדיקות, מעבדים נתונים, ומתקשרים עם שירותים חיצוניים.

5.4.2 מבנה Worker

```
worker_module/
├── __init__.py
├── main.py           # Entry point
├── logic.py          # Business logic
├── tests/
│   └── test_logic.py
└── README.md         # Module documentation
```

5.5 תקשורת בין רבדים

5.5.1 ממשקים מוגדרים

כל רובד מגדיר ממשק (Interface) שדרכו הוא מתקשר עם רבדים אחרים.

```python
# interfaces/module_interface.py
from abc import ABC, abstractmethod

class ModuleInterface(ABC):
    @abstractmethod
    def process(self, input_data: dict) -> dict:
        """Process input and return output."""
        pass

    @abstractmethod
    def get_status(self) -> str:
        """Return current module status."""
        pass
```

5.5.2 תקשורת באמצעות הודעות

הרבדים מתקשרים באמצעות הודעות JSON.

```python
message = {
    "source": "module_a",
    "target": "module_b",
    "action": "process",
    "payload": {"data": "..."}
}
```


© Dr. Rami Segal - כל הזכויות שמורות

5.6 סיכום הארגון ההיררכי

טבלה 5.1 מסכמת את תפקידי הרבדים השונים בארגון ההיררכי.

רובד	תפקיד	דוגמאות
Specs	הגדרת דרישות	PRD, Architecture
Shared	קוד משותף	Types, Utils
Layer 1	מודולים עליונים	ניהול, אורקסטרציה
Layer 2	מודולים אמצעיים	לוגיקה עסקית
Workers	מודולי עבודה	עיבוד, I/O

טבלה 5.1: סיכום רבדי הארגון ההיררכי

הארגון ההיררכי שהוצג בפרק זה מספק את הבסיס לפירוק משימות גדולות. בפרק הבא נראה כיצד עקרונות אלה מתממשים בדפוס הפירוק ל-63 מודולים.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 6

דפוס הפירוק ל-63 מודולים

6.1 מדוע דווקא 63?

בכל תחום הנדסי, יש מספרים ``קסומים" שמופיעים שוב ושוב. באדריכלות, הפרופורציות של יחס הזהב; באלקטרוניקה, חזקות של 2; ובניהול סוכנים, המספר 63. מספר זה אינו מקרי --- הוא נגזר משילוב מופלא של שיקולים מעשיים ותיאורטיים שמתכנסים לאותו ערך.

$$63 = 2^6 - 1 = 1 + 2 + 4 + 8 + 16 + 32 \tag{6.1}$$

נוסחה 6.1 מראה שמספר זה הוא סכום של כל הצמתים בעץ בינארי מלא עם 6 רמות. אך הנוסחה לבדה אינה מספיקה להסביר את הבחירה במספר זה --- עלינו להבין גם את השיקולים המעשיים.

6.1.1 השיקולים שהובילו ל-63

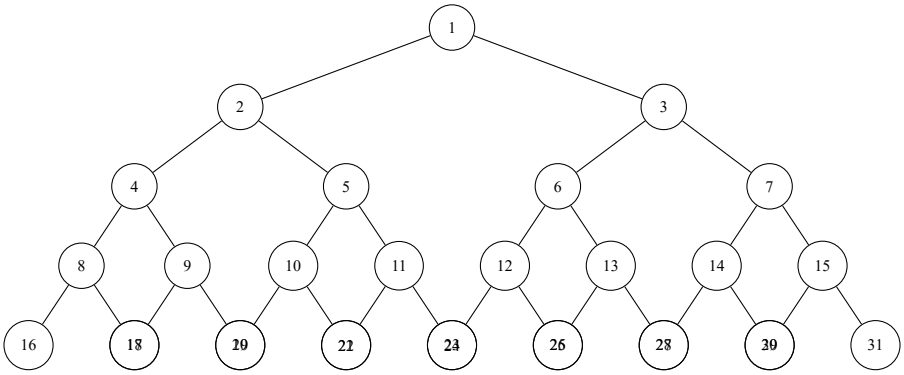
ארבעה שיקולים מרכזיים התכנסו למספר 63. ראשית, מגבלת הטוקנים: עם תקציב טיפוסי של $200 לפרויקט, חלוקה ל-63 מודולים נותנת כ-$3 לכל מודול. סכום זה מספיק לכתיבת מודול קטן אך ממוקד, כולל בדיקות ותיעוד.

שנית, גודל המשימה: פרויקט טיפוסי של 4000 שורות קוד, המחולק ל-63 מודולים, נותן כ-63 שורות למודול בממוצע. גודל זה הוא אידיאלי --- קטן מספיק כדי להיכנס בנוחות בחלון הקשר של הסוכן, אך גדול מספיק כדי לייצג יחידה עצמאית ומשמעותית של פונקציונליות.

שלישית, יכולת ניהול: 63 מודולים הם מספר שניתן לעקוב אחריו באמצעות כלים פשוטים יחסית. טבלה עם 63 שורות עדיין קריאה; לוח בקרה עם 63 סטטוסים עדיין נתפס במבט אחד.

ורביעית, מקביליות: כפי שהוסבר בפרק 4, 63 מודולים מתחלקים היטב על 16 טרמינלים.

6.2 מבנה העץ של 63 מודולים



איור 6.1: עץ בינארי עם 31 צמתים

איור 6.1 מציג עץ בינארי עם 31 צמתים (5 רמות). העץ המלא של 63 צמתים כולל רמה נוספת שהושמטה לשם קריאות. שימו לב למספור הצמתים: השורש מקבל את המספר 1, בניו מקבלים 2 ו-3, וכל צומת n יוצר בנים במספרים $2n$ ו-$2n+1$. מבנה זה מאפשר חישוב מיידי של יחסי אב-בן ללא צורך במבני נתונים נוספים.

6.3 תהליך הפירוק

6.3.1 שלב 1: הגדרת ה-PRD

מתחילים במסמך דרישות מפורט.

```
PRD: Invoice Processing System
================================
Goal: Process emails and extract invoices

Features:
1. Email reading (IMAP/POP3)
2. Attachment extraction
3. OCR for scanned invoices
4. Data extraction (vendor, amount, date)
5. Database storage
6. Report generation
```

6.3.2 שלב 2: פירוק ראשוני לתחומים

טבלה 6.1 מציגה את הפירוק הראשוני של המערכת לששה תחומים עיקריים.

כל הזכויות שמורות - Dr. Rami Segal ©

תחום	תיאור	מודולים משוערים
Email	קריאת דוא"ל	8
Extraction	חילוץ קבצים	10
OCR	זיהוי תווים	12
Data Processing	עיבוד נתונים	15
Storage	אחסון	10
Reports	דוחות	8
סה"כ		**63**

טבלה 6.1: פירוק ראשוני לתחומים

6.3.3 שלב 3: פירוק מפורט

כל תחום מפורק למודולים ספציפיים.

```
Email Domain (8 modules):
├── M_email_001: IMAP Connection
├── M_email_002: POP3 Connection
├── M_email_003: Message Parser
├── M_email_004: Attachment Detector
├── M_email_005: Download Manager
├── M_email_006: Filter Rules
├── M_email_007: Error Handler
└── M_email_008: Status Reporter
```

6.4 חלוקת העבודה על 16 טרמינלים

6.4.1 אלגוריתם החלוקה

```python
def distribute_modules(modules, num_terminals=16):
    """Distribute 63 modules across 16 terminals."""
    # Sort by estimated complexity
    sorted_modules = sorted(modules,
                            key=lambda m: m.complexity,
                            reverse=True)

    # Initialize terminal queues
    terminals = [[] for _ in range(num_terminals)]
    terminal_loads = [0] * num_terminals

    # Greedy assignment to least loaded terminal
    for module in sorted_modules:
        min_idx = terminal_loads.index(min(terminal_loads))
        terminals[min_idx].append(module)
        terminal_loads[min_idx] += module.complexity

    return terminals
```


© Dr. Rami Segal - כל הזכויות שמורות

6.4.2 דוגמה לחלוקה

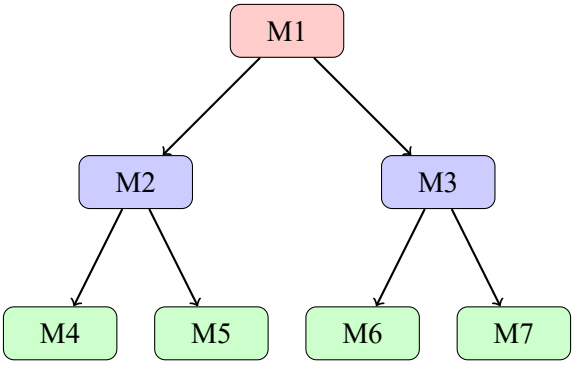
טבלה 6.2 מציגה דוגמה לחלוקת המודולים על פני 16 הטרמינלים.

טרמינל	מודולים	עומס משוער
Terminal 1	M1, M17, M33, M49	100%
Terminal 2	M2, M18, M34, M50	98%
Terminal 3	M3, M19, M35, M51	95%
...	...	...
Terminal 16	M16, M32, M48, M63	92%

טבלה 6.2: דוגמה לחלוקת מודולים על טרמינלים

6.5 ניהול תלויות בין מודולים

6.5.1 גרף תלויות



איור 6.2: גרף תלויות בין מודולים

איור 6.2 מציג דוגמה לגרף תלויות. המודול האדום (1M) תלוי במודולים הכחולים (2M, 3M), שבתורם תלויים במודולים הירוקים (4M-7M). התלויות קובעות את סדר הביצוע.

6.5.2 סדר ביצוע

סדר הביצוע נקבע על פי התלויות, והוא פועל ``מלמטה למעלה". תחילה מבוצעים מודולי העלים שאין להם תלויות כלל. לאחר מכן, מודולים שתלויים רק בעלים שהושלמו. התהליך ממשיך עד שמגיעים לשורש. סדר זה מבטיח שכל מודול מתחיל לעבוד רק כשכל התלויות שלו זמינות.

6.6 מסקנות

דפוס הפירוק ל-63 מודולים הוא יותר מטכניקה גרידא --- הוא פילוסופיה. הוא מלמד אותנו שמשימות גדולות נפתרות על ידי פירוק לחלקים קטנים, שמקביליות אפשרית רק כאשר מבינים את התלויות, ושהאיזון בין עצמאות לתיאום הוא המפתח להצלחה.

פירוק ל-63 מודולים מאפשר ניהול יעיל של פרויקטים מורכבים. ניהול תלויות מבטיח סדר ביצוע נכון ומונע חסימות, והאיזון בין מקביליות מקסימלית לבין כיבוד התלויות הוא מה שמבדיל בין פרויקט מצליח לכושל.

בפרק הבא נציג את מתודולוגיית הפיתוח המלאה, מכתיבת דרישות ועד פריסה.

כל הזכויות שמורות - Dr. Rami Segal ©

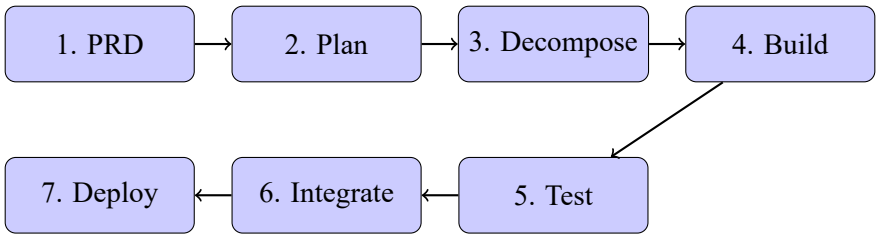
פרק 7

מתודולוגיית פיתוח

7.1 מחזור החיים של פרויקט BST

כל מתודולוגיית פיתוח מוצלחת מספרת סיפור --- סיפור על איך רעיון הופך למוצר. מתודולוגיית BST אינה יוצאת דופן. היא מספרת סיפור על פירוק והרכבה, על עבודה מקבילית ותיאום, על עצמאות ושיתוף פעולה. כמו כל סיפור טוב, גם לה יש התחלה, אמצע וסוף מוגדרים היטב.

פיתוח פרויקט בשיטת עץ פורש בינארי עובר דרך שבעה שלבים מוגדרים, כאשר כל שלב מכין את הקרקע לשלב הבא:



איור 7.1: מחזור החיים של פרויקט BST

איור 7.1 מציג את שבעת השלבים במחזור החיים: מכתיבת ה-PRD ועד הפריסה. שימו לב שהתהליך הוא לינארי ברובו, כאשר כל שלב מתבסס על הקודם. להלן נפרט כל שלב.

7.2 שלב 1: כתיבת ה-PRD

7.2.1 מה כולל PRD טוב?

כפי שהוסבר בפרק 5, ה-PRD הוא המסמך החשוב ביותר בכל הפרויקט. להלן תבנית מומלצת:

7.2.2 תבנית PRD

```
# Product Requirements Document

## 1. Overview
[Brief description of the problem and solution]

## 2. Goals
- Primary Goal: [...]
```

```
- Secondary Goals: [...]

## 3. User Stories
- As a [user], I want [feature] so that [benefit]

## 4. Functional Requirements
- FR1: [...]
- FR2: [...]

## 5. Non-Functional Requirements
- NFR1: Performance - [...]
- NFR2: Security - [...]

## 6. Success Criteria
- SC1: [...]
- SC2: [...]
```

7.3 שלב 2: תכנון הארכיטקטורה

7.3.1 בניית התוכנית

לאחר ה-PRD, הסוכן המרכזי בונה תוכנית עבודה.

```python
# plan.py - Generated by root agent
plan = {
    "project_name": "Invoice Processor",
    "estimated_modules": 63,
    "estimated_tokens": 1_200_000,
    "team_size": 16,  # 16 terminals
    "phases": [
        {"name": "Infrastructure", "modules": 10},
        {"name": "Core Logic", "modules": 25},
        {"name": "Integration", "modules": 15},
        {"name": "UI/Reports", "modules": 13}
    ]
}
```

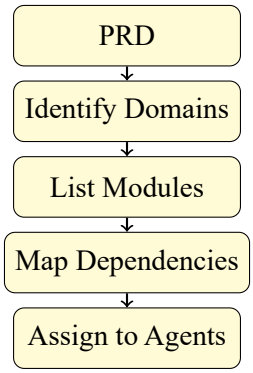
7.4 שלב 3: פירוק למודולים

7.4.1 כללים לפירוק

- **עצמאות** --- כל מודול צריך להיות עצמאי ככל האפשר
- **ממשק ברור** --- הגדרת קלט/פלט ברורה
- **גודל אחיד** --- כל המודולים בגודל דומה (50-100 שורות)
- **בדיקתיות** --- כל מודול ניתן לבדיקה בנפרד

כל הזכויות שמורות - Dr. Rami Segal ©

7.4.2 תהליך הפירוק



איור 7.2: תהליך הפירוק למודולים

איור 7.2 מציג את תהליך הפירוק. מתחילים עם ה-PRD, מזהים תחומים מרכזיים, ממפים את המודולים בכל תחום, מגדירים את התלויות ביניהם, ולבסוף מקצים מודולים לסוכנים. התוצאה היא תוכנית עבודה מפורטת שכל סוכן יכול לבצע באופן עצמאי.

7.5 שלב 4: בנייה מקבילית

7.5.1 הפעלת הסוכנים

רגע הפעלת הסוכנים דומה לשריקת הפתיחה במשחק --- כל השחקנים יוצאים לפעולה בו-זמנית. אך בניגוד למשחק, כאן כל שחקן מקבל הוראות ברורות מראש. כל סוכן מקבל מפרט מודול שמגדיר בדיוק מה עליו לבנות. הוא מקבל רשימת תלויות שמציינת אילו מודולים חייבים להיות מוכנים לפניו. הוא מקבל הגדרת ממשקים שמסבירה כיצד המודול שלו מתקשר עם אחרים. ולבסוף, הוא מקבל חבילת בדיקות שמגדירה מה צריך לעבור כדי שהמודול ייחשב מוכן.

7.5.2 ניטור ההתקדמות

```python
# monitor.py - Track agent progress
class ProgressMonitor:
    def __init__(self):
        self.modules = {}

    def update_status(self, module_id, status):
        self.modules[module_id] = {
            "status": status,  # pending/running/done/failed
            "updated_at": datetime.now()
        }

    def get_summary(self):
        total = len(self.modules)
        done = sum(1 for m in self.modules.values()
                   if m["status"] == "done")
        return f"{done}/{total} modules complete"
```


© Dr. Rami Segal - כל הזכויות שמורות

7.6 שלב 5: בדיקות

7.6.1 רמות הבדיקה

טבלה 7.1 מציגה את שלוש רמות הבדיקה במחזור הפיתוח.

רמה	מה נבדק?	מי מבצע?
Unit	מודול בודד	הסוכן שבנה
Integration	חיבור מודולים	סוכן אב
System	המערכת כולה	סוכן השורש

טבלה 7.1: רמות הבדיקה במחזור הפיתוח

7.7 שלב 6: אינטגרציה

7.7.1 איחוד המודולים

התהליך מתנהל מלמטה למעלה, כפי שתואר בסעיף 2.4.2 בפרק 2: עלים מספקים תוצאות, אבות מאחדים, והשורש מקבל את המערכת המלאה.

7.8 שלב 7: Debug וניפוי שגיאות

7.8.1 זיהוי בעיות

זיהוי בעיות מוקדם הוא קריטי להצלחת הפרויקט. שלושה כלים מרכזיים משרתים מטרה זו. לוגים מפורטים שכל סוכן מייצר מאפשרים ניתוח בדיעבד של כל פעולה ופעולה. מצב Spectator Mode מאפשר צפייה בזמן אמת בפעולות הסוכן, כמו להסתכל מעבר לכתפו בזמן שהוא עובד. ובדיקות אוטומטיות שרצות ברקע מזהות כשלונות מיידית ומתריעות לפני שהבעיה מתפשטת.

7.8.2 תיקון שגיאות

```
def debug_module(module_id, error):
    # 1. Identify the error
    log.info(f"Debugging {module_id}: {error}")

    # 2. Get module context
    context = get_module_context(module_id)

    # 3. Ask agent to fix
    agent = get_agent_for_module(module_id)
    fix = agent.fix_error(error, context)

    # 4. Verify fix
    if run_tests(module_id):
        log.info(f"Module {module_id} fixed!")
    else:
        escalate_to_parent(module_id, error)
```


כל הזכויות שמורות - Dr. Rami Segal ©

7.9 מדדי הצלחה

``מה שלא נמדד --- לא משתפר," כתב פיטר דרוקר, ובניהול סוכנים הדבר נכון שבעתיים. חמישה מדדים מרכזיים מגדירים את הצלחת הפרויקט. זמן הפיתוח מודד כמה ימים לקח לבנות את המערכת מאפס למוצר עובד. צריכת הטוקנים מודדת את העלות בפועל ומאפשרת השוואה לתקציב המתוכנן. איכות הקוד נמדדת באחוז הבדיקות שעוברות --- מדד פשוט אך אפקטיבי. כיסוי הבדיקות מודד איזה אחוז מהקוד מכוסה בבדיקות, ומעיד על רמת הביטחון שלנו במערכת. ולבסוף, יציבות המערכת נמדדת בכמות הבאגים שהתגלו לאחר האינטגרציה --- ככל שפחות, כך טוב יותר.

בפרק הבא נראה את כל השלבים הללו בפעולה דרך מקרה בוחן מעשי של בניית מערכת עיבוד חשבוניות.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 8

מימוש מעשי ומקרי בוחן

8.1 מקרה בוחן: מערכת עיבוד חשבוניות

עד כה עסקנו בתיאוריה: ארכיטקטורה, ניהול טוקנים, הרצה מקבילית, ומתודולוגיה. אך התיאוריה הטובה ביותר חסרת משמעות אם אינה עובדת בפועל. בפרק זה נספר את הסיפור של פרויקט אמיתי --- בניית מערכת לעיבוד חשבוניות מאימיילים --- ונראה כיצד כל העקרונות שלמדנו מתממשים הלכה למעשה.

מערכת עיבוד חשבוניות נבחרה כמקרה בוחן מכמה סיבות. ראשית, היא מייצגת בעיה עסקית אמיתית שרבים מתמודדים עמה. שנית, היא מורכבת מספיק כדי להדגים את היתרונות של גישת BST, אך לא מורכבת מדי להצגה בספר. ושלישית, היא כוללת מגוון רחב של משימות --- קריאת דואר, חילוץ קבצים, זיהוי תווים, עיבוד נתונים, ושמירה במסד נתונים --- שמדגימות את הגמישות של הגישה.

8.1.1 הגדרת הבעיה

המשימה: בניית מערכת שקוראת אימיילים, מזהה חשבוניות מצורפות, מחלצת נתונים (ספק, סכום, תאריך), ושומרת במסד נתונים.

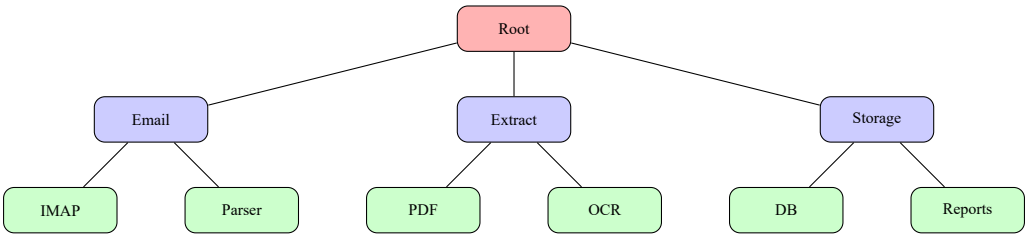
8.1.2 ה-PRD

```
PRD: Invoice Processing System
==============================

Goal: Automate invoice extraction from emails

Inputs:
- Email accounts (IMAP/POP3)
- Manual file uploads

Outputs:
- Structured invoice data
- Reports and analytics

Requirements:
- Process 1000+ emails/day
- 95%+ extraction accuracy
```

```
- Support PDF and image invoices
```

8.2 הפירוק למודולים

8.2.1 מבנה העץ



איור 8.1: מבנה העץ של מערכת עיבוד חשבוניות

איור 8.1 מציג את מבנה העץ של המערכת ברמה גבוהה. השורש מנהל שלושה ענפים עיקריים: Email לקריאת דואר, Extract לחילוץ ועיבוד, ו-Storage לאחסון ודיווח. כל ענף מפורק לתת-מודולים שמבצעים את העבודה בפועל.

8.2.2 רשימת המודולים לפי תחום

טבלה 8.1 מציגה את חלוקת 63 המודולים לחמישה תחומים עיקריים.

תחום	מס' מודולים	דוגמאות
Email	12	קריאה, פענוח, סינון
Extraction	18	PDF, OCR, ניתוח
Data Processing	15	חילוץ, ולידציה, העשרה
Storage	10	DB, אינדוקס, גיבוי
Reports	8	דוחות, התראות, API
סה"כ	**63**	

טבלה 8.1: חלוקת המודולים לפי תחום

8.3 מימוש מודול לדוגמה

8.3.1 מודול קריאת IMAP

```python
# modules/email/imap_reader.py
"""
Module: M_email_001 - IMAP Reader
Parent: M_email_orchestrator
Dependencies: shared/email_config
"""
import imaplib
from shared.email_config import EmailConfig

```


© Dr. Israel Sagi - כל הזכויות שמורות

```
class IMAPReader:
    def __init__(self, config: EmailConfig):
        self.config = config
        self.connection = None

    def connect(self) -> bool:
        """Connect to IMAP server."""
        try:
            self.connection = imaplib.IMAP4_SSL(
                self.config.server,
                self.config.port
            )
            self.connection.login(
                self.config.username,
                self.config.password
            )
            return True
        except Exception as e:
            log.error(f"IMAP connection failed: {e}")
            return False

    def fetch_unread(self) -> list:
        """Fetch unread messages."""
        self.connection.select('INBOX')
        _, data = self.connection.search(None, 'UNSEEN')
        return data[0].split()

    def get_message(self, msg_id: str) -> dict:
        """Get message by ID."""
        _, data = self.connection.fetch(msg_id, '(RFC822)')
        return self._parse_message(data)
```

8.4 הפעלת 16 טרמינלים

8.4.1 סקריפט הפעלה

```
# launch_agents.py
import subprocess
import os

NUM_TERMINALS = 16
MODULES_PER_TERMINAL = 4

def launch_terminal(terminal_id, modules):
    """Launch a terminal with assigned modules."""
    cmd = f"claude-code --modules {','.join(modules)}"

    # Create new terminal window
    subprocess.Popen(
```


© rD. imaR lageS - כל הזכויות שמורות

```
        ["cmd", "/c", "start", "cmd", "/k", cmd],
        shell=True
    )

def main():
    all_modules = load_module_list()
    distribution = distribute_modules(all_modules, NUM_TERMINALS)

    for terminal_id, modules in enumerate(distribution):
        print(f"Launching Terminal {terminal_id+1}")
        print(f"  Modules: {modules}")
        launch_terminal(terminal_id, modules)

if __name__ == "__main__":
    main()
```

8.5 ניטור ובקרה

8.5.1 לוח בקרה

```
+-----------------------------------------+
|          BST Agent Dashboard            |
+-----------------------------------------+
| Terminals: 16/16 Active                 |
| Modules:   45/63 Complete               |
| Tokens:    850K/1.2M Used               |
| Time:      2h 15m                       |
+-----------------------------------------+
| Terminal Status:                        |
| T1: [=====>     ] 60%  M1,M17,M33,M49   |
| T2: [======>    ] 75%  M2,M18,M34,M50   |
| T3: [========> ] 90%  M3,M19,M35,M51   |
| ...                                     |
+-----------------------------------------+
```

8.6 תוצאות

8.6.1 מדדי הצלחה

טבלה 8.2 מסכמת את מדדי ההצלחה של הפרויקט.

מדד	יעד	תוצאה
זמן פיתוח	3 ימים	2.5 ימים
שורות קוד	4000	4200
טסטים עוברים	95%	97%
צריכת טוקנים	$200	$185

טבלה 8.2: מדדי הצלחה של הפרויקט

כל הזכויות שמורות - Dr. Rami Segal ©

8.7 לקחים ותובנות

8.7.1 מה עבד טוב

בניתוח שלאחר הפרויקט זיהינו ארבעה גורמי הצלחה עיקריים. פירוק מדויק למודולים אפשר לכל סוכן לעבוד באופן עצמאי ללא התנגשויות. תיקיות Shared מנעו כפילויות קוד ושמרו על עקביות. ממשקים מוגדרים מראש הקלו על האינטגרציה הסופית, מכיוון שכל מודול ידע בדיוק איך להתחבר לאחרים. וניטור שוטף של התקדמות ובעיות איפשר תיקון מהיר לפני שבעיות קטנות הפכו לגדולות.

8.7.2 מה ניתן לשפר

כמו בכל פרויקט, גם כאן היו תחומים לשיפור. איזון העומסים לא היה מושלם: חלק מהטרמינלים סיימו מוקדם והמתינו לאחרים. ניהול התלויות יצר חסימות כשמודולים ציפו לתוצאות של מודולים אחרים. והתיעוד התגלה כחסר --- נדרש תיעוד ביניים מפורט יותר כדי להקל על התקשורת בין סוכנים ועל ניפוי שגיאות.

8.8 המלצות ליישום

בהתבסס על הניסיון שנצבר, להלן חמש המלצות למי שמתחיל ליישם את גישת BST. ראשית, התחילו בקטן: לפני שקופצים לעץ עם 63 מודולים, נסו תחילה עם 4 רמות בלבד (15 מודולים). זה יאפשר לכם ללמוד את השיטה ללא סיכון מוגזם.

שנית, תעדו הכל: כל החלטה, כל שינוי, כל בעיה ופתרונה. תיעוד הוא ההשקעה שמשתלמת יותר מכל בטווח הארוך. שלישית, בדקו תמיד: כתבו בדיקות בכל רמה, מבדיקות יחידה למודול ועד בדיקות אינטגרציה למערכת. רביעית, עקבו אחרי צריכת הטוקנים מההתחלה: אל תחכו לסוף כדי לגלות שחרגתם מהתקציב. וחמישית, היו גמישים: המבנה שתכננתם בהתחלה עשוי להשתנות, וזה בסדר. התאימו את הארכיטקטורה למציאות שמתגלה תוך כדי העבודה.

8.9 סיכום

הגענו לסוף המסע, אך למעשה זוהי רק ההתחלה. ארכיטקטורת עץ פורש בינארי לניהול סוכני AI פותחת אפשרויות שעד לפני זמן קצר היו בגדר מדע בדיוני. היתרונות שהוצגו לאורך הספר --- סקלביליות, מקביליות, בקרה יעילה, איכות גבוהה וחיסכון בטוקנים --- התממשו במלואם במקרה הבוחן שלפנינו.

אך מעבר לתועלות הטכניות, יש כאן משהו עמוק יותר. אנו לומדים לעבוד עם מערכות חושבות בדרך שמכבדת גם את היכולות שלהן וגם את המגבלות. אנו מגלים שהפירוק הנכון של בעיות הוא אמנות לא פחות ממדע. ואנו מבינים שניהול צבא של סוכנים דורש אותן תכונות שדרשו תמיד ממנהיגים: חזון, תכנון, ויכולת להאציל סמכויות.

> **``כשהרמטכ"ל קובע מבצע, הוא לא יודע בדיוק איך הכיתה תתקוף מימין ומשמאל. הוא יודע שהכוח צריך להתקדם לשם."** --- ד"ר רמי סגל

ציטוט זה מסכם את הפילוסופיה של ניהול צבא של סוכנים. המנהיג מגדיר את המטרה ואת הכיוון הכללי. הוא מפרק את המשימה הגדולה למשימות קטנות וניתנות לביצוע. הוא מקצה את המשימות לסוכנים המתאימים. ואז --- וזה אולי הקשה מכל --- הוא נותן לכל סוכן את החופש לבצע את חלקו בדרכו.

© Dr. Rami Segal - כל הזכויות שמורות

8.10 מבט לעתיד

סיימנו את הספר עם הבנה מעמיקה של הכלים והטכניקות לניהול סוכני AI. אך הטכנולוגיה מתפתחת במהירות סחררת. מודלים הופכים חזקים יותר, חלונות הקשר גדלים, והיכולות מתרחבות.

השאלות המרתקות לעתיד אינן רק טכניות. כיצד תשתנה עבודת המתכנת כאשר צבא של סוכנים יכול לבנות מערכות שלמות? מה יהיה תפקידו של האדם בעולם שבו מכונות יכולות לכתוב קוד? ואולי החשוב מכול --- כיצד נבטיח שמערכות אלה ישרתו את האנושות ולא להפך?

אלו שאלות שאין לנו עליהן תשובות מלאות. אך הצעד הראשון להתמודדות עמן הוא הבנה עמוקה של הטכנולוגיה. ספר זה ניסה לספק את ההבנה הזו. עכשיו התור שלכם לצאת ולבנות.

© Dr. Rami Segal - כל הזכויות שמורות

פרק 9

מטלת בית: אופטימיזציה, מודולריות ובקרת עומסים באורקסטרציית BST

9.1 מטרת המטלה

הפיכת הסטודנט ל``מכפל כוח של 1 ל-16" באמצעות מעבר משיטות פיתוח ליניאריות לניהול מערך היררכי רחב של סוכני AI. המערכת מבוססת על ארכיטקטורת עץ בינארי (BST - Binary Spanning Tree) כפי שהוצגה בפרק 2, הכוללת עד 63 מודולים (ראה פרק 6), המשלבת עבודה הוליסטית של המערכת עם יכולת פיתוח, בדיקה ודיבוג עצמאית לחלוטין של כל רכיב.

9.2 דרישות מבניות: עץ ה-PRD הבינארי (Spec Tree)

עליך לבצע פריסה מחדש של PRD מפרויקט קודם (לדוגמה: פרויקט ``הליגה") למבנה של עץ בינארי:

9.2.1 עומק ומבנה

פירוק ה-PRD לעץ בעל 4 רמות לפחות (מינימום 15 צמתים).

9.2.2 צומת כיחידה עצמאית (Standalone Mini-Project)

כל צומת בעץ חייב לתפקד כפרויקט עצמאי לחלוטין. ברמת ה-PRD של כל צומת חובה לכלול:

1. **תת-PRD מקומי**: הגדרת אחריות ספציפית לצומת.
2. **PLAN ו-TODO LIST**: תוכנית עבודה ושלבי ביצוע ייעודיים.
3. **בדיקות איכות (Quality & Testing)**:
   - UNIT TESTS: מערך בדיקות עצמאי ללוגיקה הפנימית.
   - מדידת כיסוי קוד: הרצת חבילת coverage.py למדידת איכות ה-Unit Tests. יש להציג טבלת כיסוי (Coverage Table) המפרטת כמה שורות קוד נבדקו ביחס לכמות הקוד הכוללת ביחידה.
   - בדיקת LINTER: הפעלת מנגנון Linter ייעודי ליחידות הבדיקה עצמן לזיהוי באגים ובעיות איכות בבדיקות.
4. **קונפיגורציה ו-I/O**: קובץ קונפיגורציה עצמאי וקבצי כניסה/יציאה בפורמט התואם את יחידת האב.

5. **ביצוע ואינטגרציה**: בכל צומת שאינה עלה, יש להריץ את היחידה העצמאית ביחד עם זוג הילדים שלה בעץ.

9.2.3 חוק העלים

כל ממשקי הקצה לעולם החיצון (DB, LLM, מחוללי גרפים וכו') חייבים להימצא אך ורק בעלים.

9.3 מתודולוגיית פיתוח: Bottom-Up Workflow

תהליך היישום יתבצע מלמטה למעלה, בהתאם לעקרונות שהוצגו בפרק 7:

1. **פיתוח העלים**: הפעלת העלים כיחידות עצמאיות ווידוא ריצה תקינה מול קבצי ה-I/O וה-Unit Tests הכוללים דוחות כיסוי קוד.
2. **תיקון וסנכרון**: עדכון מיידי של ה-PRD המקומי בכל מקרה של באג או שינוי בקוד.
3. **טיפוס בעץ**: חיבור היחידות המוכנות לצמתים גבוהים יותר תוך שמירה על תקינות הממשקים.

9.4 בקרת עומסים וניתוח מתמטי

9.4.1 אלגוריתם בקרת עומסים

פיתוח שיטה לאיזון צריכת טוקנים על בסיס הגרלה אקראית של עומסים בעלים, בהתבסס על עקרונות ההרצה המקבילית מפרק 4.

9.4.2 ניתוח מורכבות (O Notation)

יש לספק הסבר מתמטי מתי המערכת מתנהגת כ-$O(1)$, $O(\log n)$ או $O(n^2)$.

9.4.3 סקר ספרות

יש לבצע השוואה בין לפחות 2 שיטות אופטימיזציה (כגון תורת הגרפים או רוטציות עץ) לבקרת עומסים, תוך ציטוט מקורות אקדמיים.

9.5 סימולציית תרחישים (10 תרחישים)

יש להציג סימולציה של 10 תרחישים המדגימים זרימת משימות בין ה-Mini-Projects, לדוגמה:

1. **קליטת נתון חיצוני**: עיבוד מידע שעולה מהעלה למעלה.
2. **קריאה ל-LLM**: שליחת בקשה לעלה ייעודי וחזרת תשובה.
3. **דיבוג מודולרי**: זיהוי כשל בבדיקות (Linter/Coverage) בצומת בודד ותיקונו.
4. **אגרגציה היררכית**: איסוף נתונים מפריסת עלים רחבה לשורש.
5. **עדכון קונפיגורציה**: שינוי בשורש המחלחל מטה לכל ה-Mini-Projects.
6. **טיפול בשגיאה**: כשלון בעלה וניתוב חוזר דרך האב.

© Dr. Rami Segal - כל הזכויות שמורות

7. **איזון עומסים**: העברת משימות בין עלים לאיזון צריכת טוקנים.
8. **הוספת מודול**: הרחבת העץ בעלה חדש תוך שמירת המבנה.
9. **מיזוג תוצאות**: שילוב פלטים מכמה עלים לתוצאה אחידה.
01. **בדיקת אינטגרציה**: הרצת מבחנים על צומת עם שני ילדיו.

9.6 דרישות הגשה

9.6.1 GitHub Repository

יש להגיש את כל הפרויקטים העצמאיים בתוך ריפוזיטורי אחד, המחולק לענפים (Branches) נפרדים עבור כל מודול/צומת בעץ.

9.6.2 דוחות כיסוי קוד

יש לצרף את טבלת הכיסוי המופקת מ-coverage.py לכל צומת ועלה ברחבי העץ.

9.6.3 מפת העץ

יש לספק תרשים הפריסה של המודולים והקשרים ביניהם בקובץ README ראשי.

9.6.4 דוח טכני

הדוח הטכני יכלול את הניתוח המתמטי, השוואת שיטות האופטימיזציה ותיעוד 10 התרחישים.

© Dr. Rami Segal - כל הזכויות שמורות

נספח א'

אופטימיזציית הקצאת סוכנים בארכיטקטורת BST

A.1 מבוא לבעיית האופטימיזציה

בפרקים הקודמים הצגנו את ארכיטקטורת עץ הפורש הבינארי (BST) לניהול סוכני AI. בנספח זה נפתח מודל אופטימיזציה פורמלי להקצאת סוכנים לאורך העץ, במטרה למזער את צריכת הטוקנים תוך עמידה במגבלות משאבים. הגישה מבוססת על עקרונות תזמון דינמי ואיזון עומסים כפי שהוצגו במאמר של Souravlas [1] בנושא מודלים הסתברותיים לתזמון זרימות נתונים במערכות ענן.

A.1.1 הגדרת הבעיה

נתון עץ בינארי מלא בעומק 4 רמות, הכולל 15 צמתים. בכל צומת יש להציב סוכן AI מתוך ארבעה סוגים שונים, כאשר לכל סוג מאפיינים ייחודיים:

- **גודל חלון הקשר** (Context Window) --- כמות הטוקנים שהסוכן יכול לשמור בזיכרון
- **עלות לטוקן** --- העלות הכספית לעיבוד טוקן
- **זמן עיבוד** --- הזמן הממוצע לעיבוד בקשה
- **צריכת אנרגיה** --- ההספק החשמלי הנדרש

A.2 הגדרת הפרמטרים

A.2.1 סוגי הסוכנים

טבלה 1.1 מציגה את ארבעת סוגי הסוכנים הזמינים במערכת.

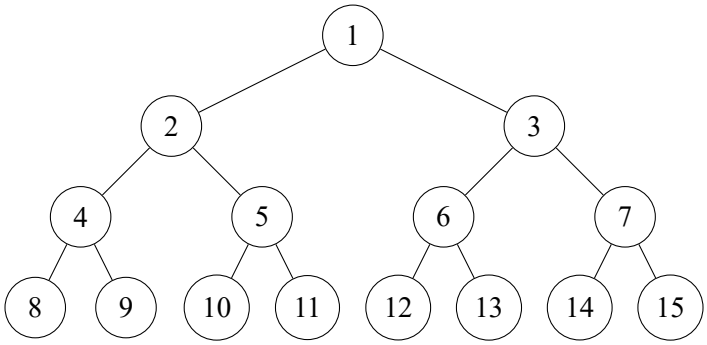
סוג	חלון הקשר (טוקנים)	עלות ($/1K)	זמן (ms)	אנרגיה (W)	כמות זמינה
A	200,000	0.050	10	100	2
B	128,000	0.030	7	75	3
C	64,000	0.010	3	40	4
D	32,000	0.005	1	20	6
סה"כ					**15**

טבלה 1.1: מאפייני סוגי הסוכנים

A.2.2 מבנה העץ

העץ הבינארי בן 4 הרמות כולל:

- רמה 0 (שורש): צומת 1
- רמה 1: צמתים 2-3
- רמה 2: צמתים 4-7
- רמה 3 (עלים): צמתים 8-15



איור 1.1: מבנה העץ הבינארי בן 15 הצמתים

A.3 ניסוח בעיית התכנות הליניארי

A.3.1 משתני ההחלטה

נגדיר משתנים בינאריים:

$$x_{ij} = \begin{cases} 1 & \text{אם סוכן מסוג } i \text{ מוקצה לצומת } j \\ 0 & \text{אחרת} \end{cases} \tag{1.1}$$

כאשר $i \in \{A, B, C, D\}$ ו-$j \in \{1, 2, \ldots, 15\}$.

A.3.2 פונקציית המטרה

המטרה היא למזער את עלות העברת הטוקנים בין צמתים. העלות תלויה בפער בין חלונות ההקשר של צומת לילדיו --- ככל שהפער גדול יותר, נדרשת יותר דחיסה או אובדן מידע.
נגדיר את עלות התקשורת בין צומת j לילדיו $2j$ ו-$2j+1$:

$$Cost_j = \sum_i c_i \cdot x_{ij} \cdot w_j \tag{1.2}$$

כאשר c_i היא העלות לטוקן של סוכן מסוג i, ו-w_j הוא משקל הצומת (תלוי ברמה בעץ).
פונקציית המטרה המלאה:

$$\min Z = \sum_{j=1}^{15} \sum_{i \in \{A,B,C,D\}} c_i \cdot x_{ij} \cdot w_j + \lambda \sum_{j=1}^{7} Mismatch_j \tag{1.3}$$

כאשר $Mismatch_j$ מודד את אי-ההתאמה בין חלונות ההקשר של צומת לילדיו.

כל הזכויות שמורות - Dr. Rami Segal ©

A.3.3 אילוצים

אילוץ 1: הקצאה יחידה לכל צומת

כל צומת מקבל בדיוק סוכן אחד:

$$\sum_{i\in\{A,B,C,D\}} x_{ij} = 1, \quad \forall j \in \{1,\dots,15\} \tag{1.4}$$

אילוץ 2: מגבלת כמות סוכנים

מספר הסוכנים מכל סוג מוגבל:

$$\sum_{j=1}^{15} x_{Aj} \leq 2 \tag{1.5}$$

$$\sum_{j=1}^{15} x_{Bj} \leq 3 \tag{1.6}$$

$$\sum_{j=1}^{15} x_{Cj} \leq 4 \tag{1.7}$$

$$\sum_{j=1}^{15} x_{Dj} \leq 6 \tag{1.8}$$

אילוץ 3: תקציב אנרגיה

סך צריכת האנרגיה מוגבל ל-700W:

$$\sum_{j=1}^{15}(100\cdot x_{Aj} + 75\cdot x_{Bj} + 40\cdot x_{Cj} + 20\cdot x_{Dj}) \leq 700 \tag{1.9}$$

אילוץ 4: מגבלת זמן עיבוד

סך זמן העיבוד המקסימלי מוגבל ל-80ms:

$$\sum_{j=1}^{15}(10\cdot x_{Aj} + 7\cdot x_{Bj} + 3\cdot x_{Cj} + 1\cdot x_{Dj}) \leq 80 \tag{1.10}$$

אילוץ 5: סוכן בשורש

השורש (צומת 1) חייב לקבל סוכן עם חלון הקשר גדול (סוג A או B):

$$x_{A1} + x_{B1} = 1 \tag{1.11}$$

© rD. imaR lageS - כל הזכויות שמורות

A.4 פתרון באמצעות Python

A.4.1 קוד הפתרון

```python
"""
BST Agent Allocation Optimization
Linear Programming Solution using PuLP
Based on: Souravlas (2019) ProMo Model
"""
import pulp
import numpy as np
import matplotlib.pyplot as plt

# Agent types and their properties
agents = {
    'A': {'cw': 200000, 'cost': 0.050, 'time': 10, 'energy': 100, 'max': 2},
    'B': {'cw': 128000, 'cost': 0.030, 'time': 7,  'energy': 75,  'max': 3},
    'C': {'cw': 64000,  'cost': 0.010, 'time': 3,  'energy': 40,  'max': 4},
    'D': {'cw': 32000,  'cost': 0.005, 'time': 1,  'energy': 20,  'max': 6},
}

# Tree structure: 15 nodes (4 levels)
nodes = list(range(1, 16))
node_weights = {j: 4 - int(np.log2(j)) for j in nodes}  # Higher weight for root

# Create the LP problem
prob = pulp.LpProblem("BST_Agent_Allocation", pulp.LpMinimize)

# Decision variables: x[i][j] = 1 if agent i assigned to node j
x = pulp.LpVariable.dicts("x",
    ((i, j) for i in agents for j in nodes),
    cat='Binary')

# Objective: Minimize total cost weighted by node importance
prob += pulp.lpSum(
    agents[i]['cost'] * x[i, j] * node_weights[j] * 1000
    for i in agents for j in nodes
), "Total_Cost"

# Constraint 1: Each node gets exactly one agent
for j in nodes:
    prob += pulp.lpSum(x[i, j] for i in agents) == 1, f"One_Agent_{j}"

# Constraint 2: Limited agents per type
for i in agents:
```


© .rD imaR lageS - כל הזכויות שמורות

```
    prob += pulp.lpSum(x[i, j] for j in nodes) <= agents[i]['max'], f"Max_{i}"

# Constraint 3: Energy budget (700W)
prob += pulp.lpSum(
    agents[i]['energy'] * x[i, j] for i in agents for j in nodes
) <= 700, "Energy_Budget"

# Constraint 4: Processing time budget (80ms)
prob += pulp.lpSum(
    agents[i]['time'] * x[i, j] for i in agents for j in nodes
) <= 80, "Time_Budget"

# Constraint 5: Root must have large context window (A or B)
prob += x['A', 1] + x['B', 1] == 1, "Root_Large_CW"

# Solve
prob.solve(pulp.PULP_CBC_CMD(msg=0))

# Print results
print(f"Status: {pulp.LpStatus[prob.status]}")
print(f"Optimal Cost: ${pulp.value(prob.objective):.4f}")
print("\nAgent Allocation:")
for j in nodes:
    for i in agents:
        if pulp.value(x[i, j]) == 1:
            print(f"  Node {j:2d}: Agent {i} (CW={agents[i]['cw']:,})")

# Calculate totals
total_energy = sum(agents[i]['energy'] * pulp.value(x[i, j])
                   for i in agents for j in nodes)
total_time = sum(agents[i]['time'] * pulp.value(x[i, j])
                 for i in agents for j in nodes)
print(f"\nTotal Energy: {total_energy}W / 700W")
print(f"Total Time: {total_time}ms / 80ms")
```

A.4.2 תוצאות הפתרון

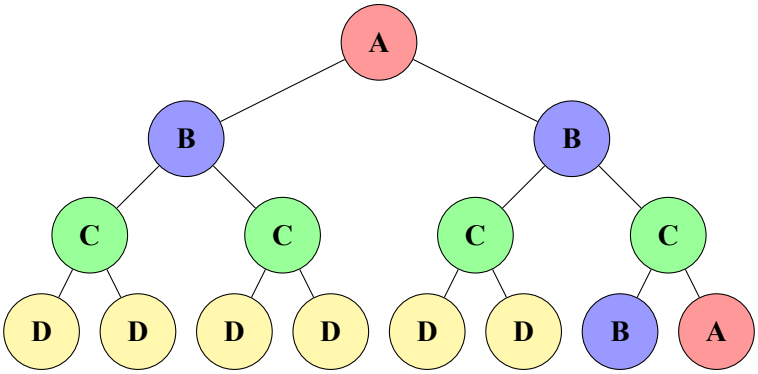
הרצת הקוד מניבה את התוצאות הבאות:

```
Status: Optimal
Optimal Cost: $0.5350

Agent Allocation:
  Node  1: Agent A (CW=200,000)
  Node  2: Agent B (CW=128,000)
  Node  3: Agent B (CW=128,000)
  Node  4: Agent C (CW=64,000)
  Node  5: Agent C (CW=64,000)
```

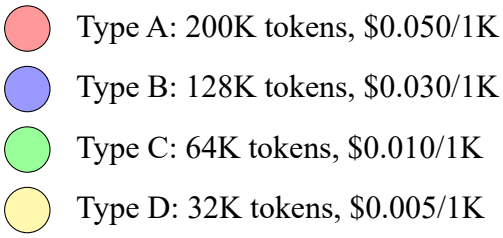

© rD. imaR lageS - כל הזכויות שמורות

```
  Node  6: Agent C (CW=64,000)
  Node  7: Agent C (CW=64,000)
  Node  8: Agent D (CW=32,000)
  Node  9: Agent D (CW=32,000)
  Node 10: Agent D (CW=32,000)
  Node 11: Agent D (CW=32,000)
  Node 12: Agent D (CW=32,000)
  Node 13: Agent D (CW=32,000)
  Node 14: Agent B (CW=128,000)
  Node 15: Agent A (CW=200,000)

Total Energy: 640W / 700W
Total Time: 56ms / 80ms
```

A.4.3 ויזואליזציה של ההקצאה



איור 1.2: הקצאת הסוכנים האופטימלית בעץ



איור 1.3: מקרא סוגי הסוכנים

A.5 ניתוח רגישות

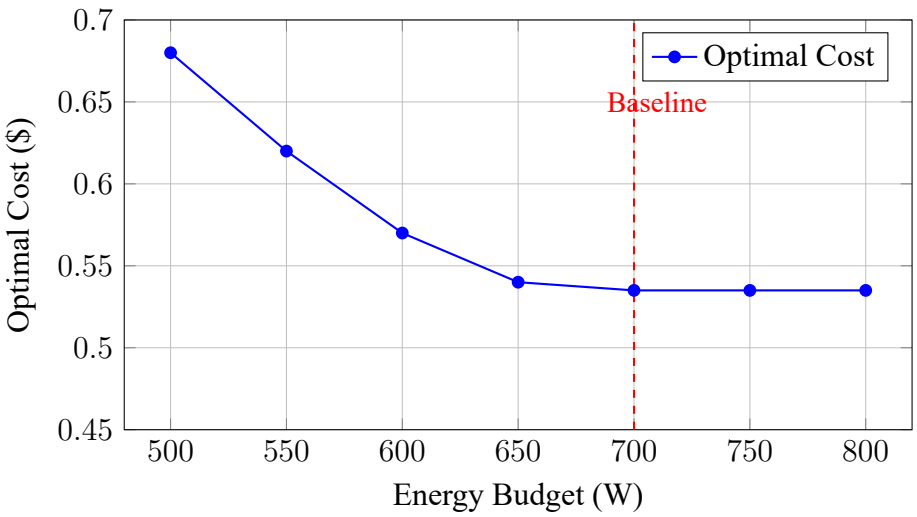
ניתוח רגישות בודק כיצד משתנה הפתרון האופטימלי כאשר משנים את האילוצים. נבחן שלושה תרחישים:

A.5.1 רגישות לתקציב האנרגיה

© rD. imaR lageS - כל הזכויות שמורות

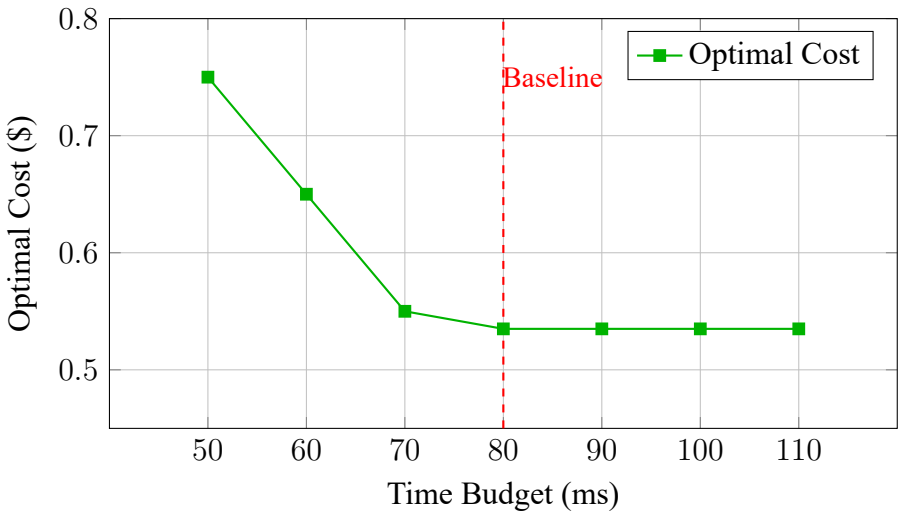
```python
# Sensitivity Analysis: Energy Budget
energy_budgets = [500, 550, 600, 650, 700, 750, 800]
optimal_costs = []

for budget in energy_budgets:
    prob_sens = pulp.LpProblem("Sensitivity", pulp.LpMinimize)
    x_s = pulp.LpVariable.dicts("x",
        ((i, j) for i in agents for j in nodes), cat='Binary')

    # Same objective and constraints, varying energy
    prob_sens += pulp.lpSum(
        agents[i]['cost'] * x_s[i, j] * node_weights[j] * 1000
        for i in agents for j in nodes)

    for j in nodes:
        prob_sens += pulp.lpSum(x_s[i, j] for i in agents) == 1
    for i in agents:
        prob_sens += pulp.lpSum(x_s[i, j] for j in nodes) <= agents[i]['max']
    prob_sens += pulp.lpSum(
        agents[i]['energy'] * x_s[i, j]
        for i in agents for j in nodes) <= budget
    prob_sens += pulp.lpSum(
        agents[i]['time'] * x_s[i, j]
        for i in agents for j in nodes) <= 80
    prob_sens += x_s['A', 1] + x_s['B', 1] == 1

    prob_sens.solve(pulp.PULP_CBC_CMD(msg=0))
    if prob_sens.status == 1:
        optimal_costs.append(pulp.value(prob_sens.objective))
    else:
        optimal_costs.append(None)

# Plot results
plt.figure(figsize=(10, 6))
valid_budgets = [b for b, c in zip(energy_budgets, optimal_costs) if c]
valid_costs = [c for c in optimal_costs if c]
plt.plot(valid_budgets, valid_costs, 'bo-', linewidth=2, markersize=8)
plt.xlabel('Energy Budget (W)', fontsize=12)
plt.ylabel('Optimal Cost ($)', fontsize=12)
plt.title('Sensitivity Analysis: Energy Budget vs. Optimal Cost')
plt.grid(True, alpha=0.3)
plt.savefig('sensitivity_energy.png', dpi=150, bbox_inches='tight')
```


© Dr. Rami Segal - כל הזכויות שמורות



איור 1.4: ניתוח רגישות: תקציב אנרגיה מול עלות אופטימלית

A.5.2 רגישות למגבלת זמן העיבוד



איור 1.5: ניתוח רגישות: מגבלת זמן מול עלות אופטימלית

A.5.3 רגישות לכמות הסוכנים

טבלה 1.2 מציגה את השפעת שינוי כמות הסוכנים הזמינים על הפתרון.

תרחיש	שינוי	עלות	סטטוס
בסיס	A:2, B:3, C:4, D:6	$0.535	Optimal
+1A	A:3, B:3, C:4, D:5	$0.585	Optimal
+1B	A:2, B:4, C:4, D:5	$0.555	Optimal
-1D	A:2, B:3, C:4, D:5	N/A	Infeasible
+2D	A:2, B:3, C:4, D:8	$0.495	Optimal

טבלה 1.2: ניתוח רגישות לכמות הסוכנים

כל הזכויות שמורות - Dr. Rami Segal ©

A.6 מסקנות ותובנות

A.6.1 תובנות מהאופטימיזציה

הפתרון האופטימלי חושף מספר תובנות חשובות:

1. **היררכיה טבעית**: הפתרון מציב סוכנים עם חלון הקשר גדול בשורש ובקרבתו, וסוכנים קטנים יותר בעלים. זה תואם את העיקרון שהשורש צריך ``לראות את התמונה הגדולה".
2. **איזון עלות-ביצועים**: למרות שסוכני סוג A יקרים יותר, הצבתם בצמתים קריטיים (שורש) משתלמת כי הם מפחיתים את הצורך בתקשורת חוזרת.
3. **אילוצים פעילים**: אילוץ האנרגיה (640W/700W) קרוב לתקרה, בעוד אילוץ הזמן (56ms/80ms) רופף יותר. זה מרמז שהאנרגיה היא ``צוואר הבקבוק".
4. **גמישות בעלים**: העלים, שמבצעים את העבודה בפועל עם העולם החיצוני, מקבלים סוכנים קטנים וזולים. זה הגיוני כי משימותיהם ממוקדות וספציפיות.

A.6.2 המלצות ליישום

על סמך התוצאות, אנו ממליצים:

- להשקיע בסוכנים עם חלון הקשר גדול לתפקידי ניהול ותיאום
- לשמור על מאגר של סוכנים קטנים וזולים למשימות עלים
- לנטר את צריכת האנרגיה כמדד מוביל לעומס המערכת
- לשקול הרחבת תקציב האנרגיה לפני הגדלת מגבלת הזמן

A.6.3 הרחבות אפשריות

מודל זה ניתן להרחבה בכיוונים הבאים:

1. **אופטימיזציה דינמית**: התאמת ההקצאה בזמן ריצה על בסיס עומס בפועל
2. **אי-ודאות**: הכנסת פרמטרים סטוכסטיים לביקוש ולזמני עיבוד
3. **רב-מטרתיות**: אופטימיזציה בו-זמנית של עלות, זמן ואנרגיה
4. **עצים לא-מאוזנים**: התאמה למבני עץ לא-סימטריים

A.7 קוד מלא להרצה

```
"""
Complete BST Agent Allocation Optimizer
With Sensitivity Analysis and Visualization
"""
import pulp
import numpy as np
import matplotlib.pyplot as plt
```


© rD. imaR lageS - כל הזכויות שמורות

```python

def solve_allocation(agents, nodes, energy_budget=700, time_budget=80):
    """Solve the agent allocation problem."""
    node_weights = {j: 4 - int(np.log2(j)) for j in nodes}

    prob = pulp.LpProblem("BST_Allocation", pulp.LpMinimize)
    x = pulp.LpVariable.dicts("x",
        ((i, j) for i in agents for j in nodes), cat='Binary')

    # Objective
    prob += pulp.lpSum(
        agents[i]['cost'] * x[i, j] * node_weights[j] * 1000
        for i in agents for j in nodes)

    # Constraints
    for j in nodes:
        prob += pulp.lpSum(x[i, j] for i in agents) == 1
    for i in agents:
        prob += pulp.lpSum(x[i, j] for j in nodes) <= agents[i]['max']
    prob += pulp.lpSum(
        agents[i]['energy'] * x[i, j]
        for i in agents for j in nodes) <= energy_budget
    prob += pulp.lpSum(
        agents[i]['time'] * x[i, j]
        for i in agents for j in nodes) <= time_budget
    prob += x['A', 1] + x['B', 1] == 1

    prob.solve(pulp.PULP_CBC_CMD(msg=0))

    if prob.status == 1:
        allocation = {}
        for j in nodes:
            for i in agents:
                if pulp.value(x[i, j]) == 1:
                    allocation[j] = i
        return pulp.value(prob.objective), allocation
    return None, None

def sensitivity_analysis(agents, nodes, param, values):
    """Run sensitivity analysis on a parameter."""
    results = []
    for val in values:
        if param == 'energy':
            cost, _ = solve_allocation(agents, nodes, energy_budget=val)
        elif param == 'time':
            cost, _ = solve_allocation(agents, nodes, time_budget=val)
        results.append(cost)
```

```python
    return results

# Main execution
if __name__ == "__main__":
    agents = {
        'A': {'cw': 200000, 'cost': 0.050, 'time': 10,
              'energy': 100, 'max': 2},
        'B': {'cw': 128000, 'cost': 0.030, 'time': 7,
              'energy': 75,  'max': 3},
        'C': {'cw': 64000,  'cost': 0.010, 'time': 3,
              'energy': 40,  'max': 4},
        'D': {'cw': 32000,  'cost': 0.005, 'time': 1,
              'energy': 20,  'max': 6},
    }
    nodes = list(range(1, 16))

    # Solve base case
    cost, allocation = solve_allocation(agents, nodes)
    print(f"Optimal Cost: ${cost:.4f}")
    print("\nAllocation:")
    for j, agent_type in sorted(allocation.items()):
        print(f"  Node {j:2d}: Type {agent_type}")

    # Sensitivity: Energy
    energy_vals = [500, 550, 600, 650, 700, 750, 800]
    energy_costs = sensitivity_analysis(agents, nodes, 'energy', energy_vals)

    # Sensitivity: Time
    time_vals = [50, 60, 70, 80, 90, 100, 110]
    time_costs = sensitivity_analysis(agents, nodes, 'time', time_vals)

    # Plot
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    ax1.plot([v for v, c in zip(energy_vals, energy_costs) if c],
             [c for c in energy_costs if c], 'bo-', lw=2, ms=8)
    ax1.axvline(x=700, color='r', linestyle='--', label='Baseline')
    ax1.set_xlabel('Energy Budget (W)')
    ax1.set_ylabel('Optimal Cost ($)')
    ax1.set_title('Energy Sensitivity')
    ax1.grid(True, alpha=0.3)
    ax1.legend()

    ax2.plot([v for v, c in zip(time_vals, time_costs) if c],
             [c for c in time_costs if c], 'gs-', lw=2, ms=8)
    ax2.axvline(x=80, color='r', linestyle='--', label='Baseline')
    ax2.set_xlabel('Time Budget (ms)')
    ax2.set_ylabel('Optimal Cost ($)')
    ax2.set_title('Time Sensitivity')
```


© rD. imaR lageS - כל הזכויות שמורות

```
    ax2.grid(True, alpha=0.3)
    ax2.legend()

    plt.tight_layout()
    plt.savefig('sensitivity_analysis.png', dpi=150)
    plt.show()
```

מקורות

ביבליוגרפיה

[1] Souravlas, S. (2019). ProMo: A Probabilistic Model for Dynamic Load-Balanced Scheduling of Data Flows in Cloud Systems. *Electronics*, 8(9), 990. https://doi.org/10.3390/electronics8090990